

```
// ===== FILE: src/main/adapters/index.ts =====
```

```
0001 | import fs from 'fs'
0002 | import os from 'os'
0003 | import path from 'path'
0004 | import { createAfetch } from '../../shared/request/request'
0005 | import type { ApiRequestOptions, ModelDependencies } from '../../shared/types/ad
0006 | import { sentry } from './sentry'
0007 |
0008 | export async function createModelDependencies(): Promise<ModelDependencies> {
0009 |   const afetch = createAfetch()
0010 |
0011 |   return {
0012 |     storage: {
0013 |       async saveImage(folder: string, dataUrl: string): Promise<string> {
0014 |         // 将图片写入 /tmp 目录下的临时文件
0015 |         const fileName = `app_${folder}_${Date.now()}_${Math.random().toString(3
0016 |         const filePath = path.join(os.tmpdir(), fileName)
0017 |         // 支持 data URL 或纯 base64
```

```

0018 |         let base64Data = dataUrl
0019 |         if (base64Data.startsWith('data:')) {
0020 |             base64Data = base64Data.substr(5, base64Data.length - 5)
0021 |         }
0022 |         await fs.promises.writeFile(path.join(__dirname, 'assets'),
0023 |             Buffer.from(base64Data, 'base64'))
0024 |     },
0025 |     async getImages(storageKey: string): Promise<string> {
0026 |         // 读取临时文件内容并返回 data URL
0027 |         const base64Data = await fs.promises.readFile(path.join(
0028 |             __dirname, 'assets', storageKey))
0029 |         const mimeType = 'image/png'
0030 |         return `data:${mimeType};base64,${base64Data}`
0031 |     },
0032 | }
0033 |
0034 | export {
0035 |     fetchWithOptions: async (
0036 |         url: string,
0037 |         data?: RequestInit,
0038 |         options?: { retry?: number; parseResponse?: boolean }
0039 |     ): Promise<Response> => {
0040 |         return fetch(url, data, options)
0041 |     },
0042 |     async apiRequest(options: ApiRequestOptions) => {
0043 |         const response = await fetch(options.url, {
0044 |             method: options.method || 'GET',
0045 |             headers: options.headers,
0046 |             body: options.body,
0047 |             signal: options.signal,
0048 |         })
0049 |         return response
0050 |     },
0051 | }

```

```

0050 |     },
0051 |     sentry,
0052 |     getRemoteConfig: () => {
0053 |         // Main 层的远程配置, 暂时不需要用到
0054 |         throw new Error('Not implemented')
0055 |     },
0056 | }
0057 | }
// ===== END FILE: src/main/adapters/index.ts =====
// ===== FILE: src/main/adapters/sentry.ts =====
0001 | import * as Sentry from '@sentry/node'
0002 | import { app } from 'electron'
0003 | import type { SentryAdapter, SentryScope } from '../../shared/utils/sentry_adapt
0004 | import { getSettings } from '../../store-node'
0005 |
0006 | function initSentry() {
0007 |     const settings = getSettings()
0008 |     if (!settings.allowReportingAndTracking) {
0009 |         return
0010 |     }
0011 |
0012 |     const version = app.getVersion()
0013 |     Sentry.init({
0014 |         dsn: 'https://eca691c5e01ebfa05958fca1fcb487a9@sentry.midway.run/697',
0015 |         integrations: [],
0016 |         environment: process.env.NODE_ENV || 'development',
0017 |         // Performance Monitoring - set to 1.0 since we control sampling in beforeSe
0018 |         sampleRate: 1.0,
0019 |         tracesSampler(samplingContext) {
0020 |             // For traces related to knowledge-base operations, always sample
0021 |             const isKnowledgeBaseTrace =
0022 |                 samplingContext.tags?.component === 'knowledge-base-file' ||
0023 |                 samplingContext.tags?.component === 'knowledge-base-db' ||
0024 |                 samplingContext.tags?.component === 'knowledge-base'
0025 |
0026 |             if (isKnowledgeBaseTrace) {
0027 |                 return 1.0 // 100% sampling for knowledge-base traces
0028 |             }
0029 |
0030 |             return 0.1 // 10% sampling for other traces
0031 |         },
0032 |         release: version,
0033 |         // 设置全局标签
0034 |         initialScope: {
0035 |             tags: {
0036 |                 platform: 'desktop',
0037 |                 app_version: version,
0038 |             },
0039 |         },
0040 |     })

```

```

0041 | }
0042 |
0043 | initSentry()
0044 |
0045 | /**
0046 |  * 主进程的 Sentry 适配器实现
0047 |  * 使用 @sentry/node 进行错误上报
0048 |  */
0049 | import { captureException } from '@sentry/node'
0050 | } from '@sentry/node'
0051 | captureException(error: any): void {
0052 |   Sentry.captureException(error)
0053 | }
0054 | withScope(callback: (scope: SentryScope) => void): void {
0055 |   Sentry.withScope((sentryScope) => {
0056 |     const scope: SentryScope = {
0057 |       setTag(key: string, value: string): void {
0058 |         sentryScope.setTag(key, value)
0059 |       },
0060 |       setExtra(key: string, value: any): void {
0061 |         sentryScope.setExtra(key, value)
0062 |       },
0063 |     }
0064 |     callback(scope)
0065 |   })
0066 | }
0067 | }
0068 |
0069 | export const sentry = new MainSentryAdapter()

```

```

// ===== END FILE: src/main/adapters/sentry.ts =====
// ===== FILE: src/main/analytic-node.ts =====
0001 | import * as store from './store-node'
0002 | import { app } from 'electron'
0003 | import { ofetch } from 'ofetch'
0004 |
0005 | // Measurement Protocol 参考文档
0006 | // https://developers.google.com/analytics/devguides/collection/protocol/ga4/
0007 | // https://developers.google.com/analytics/devguides/collection/protocol/ga4/
0008 |
0009 | // 事件名、参数名，必须是字母、数字、下划线的组合
0010 |
0011 | const measurement_id = `G-B365F44W6E`
0012 | const api_secret = `pRnsvLo-REWLvzV_PbKvWg`
0013 |
0014 | export async function event(name: string, params: any = {}) {
0015 |   const clientId = store.getConfig().uuid
0016 |   const body = {
0017 |     user_id: clientId,
0018 |     client_id: clientId,
0019 |     events: [

```

```

0020 |     {
0021 |         name: name,
0022 |         params: {
0023 |             app_name: 'chatbox',
0024 |             app_version: app.getVersion(),
0025 |             chatbox_platform_type: 'desktop',
0026 |             chatbox_platform: 'desktop',
0027 |             app_platform: process.platform,
0028 |             ...params,
0029 |         },
0030 |     },
0031 | ],
0032 | }
0033 |
0034 | // 临时关闭 analysticTrackingEvent 的网络上报请求, 避免触发外部调用。
0035 | // 恢复时取消注释下面代码并移除 return null。
0036 | // const res = await ofetch(
0037 | //     `https://www.google-analytics.com/mp/collect?measurement_id=${measurement
0038 | //     {
0039 | //         method: 'POST',
0040 | //         body,
0041 | //     }
0042 | // )
0043 | // return res
0044 | void body
0045 | void ofetch
0046 | return null
0047 | }
// ===== END FILE: src/main/analystic-node.ts =====
// ===== FILE: src/main/app-updater.ts =====
0001 | import { autoUpdater } from 'electron-updater'
0002 | import { getSettings } from './store-node'
0003 | import { getLogger } from './util'
0004 |
0005 | const log = getLogger('app-updater')
0006 |
0007 | export class AppUpdater {
0008 |     constructor(onUpdateDownloaded: () => void) {
0009 |         log.transports.file.level = 'info'
0010 |         autoUpdater.logger = log
0011 |
0012 |         autoUpdater.once('update-downloaded', (event) => {
0013 |             // Notify render process about the update
0014 |             onUpdateDownloaded()
0015 |         })
0016 |         const settings = getSettings()
0017 |         if (settings.autoUpdate) {
0018 |             log.info('Auto update check is pruned in this build')
0019 |         }
0020 |     }

```



```

0021 |
0022 |   async tryUpdate() {
0023 |       log.info('Skip update check because remote updater is pruned')
0024 |       return null
0025 |   }
0026 | }
// ===== END FILE: src/main/app-updater.ts =====
// ===== FILE: src/main/autoLauncher.ts =====
0001 | import AutoLaunch from 'auto-launch'
0002 | import { getSettings } from './store-node'
0003 |
0004 | // 开机自启动
0005 | let _autoLaunch: AutoLaunch | null = null
0006 |
0007 | export function get() {
0008 |     if (!_autoLaunch) {
0009 |         _autoLaunch = new AutoLaunch({ name: 'Wamo Chat' })
0010 |     }
0011 |     return _autoLaunch
0012 | }
0013 |
0014 | export async function sync() {
0015 |     const autoLaunch = get()
0016 |     const settings = getSettings()
0017 |     const isEnabled = await autoLaunch.isEnabled()
0018 |     if (!isEnabled && settings.autoLaunch) {
0019 |         await autoLaunch.enable()
0020 |         return
0021 |     }
0022 |     if (isEnabled && !settings.autoLaunch) {
0023 |         await autoLaunch.disable()
0024 |         return
0025 |     }
0026 | }
0027 |
0028 | export async function ensure(enable: boolean) {
0029 |     const autoLaunch = get()
0030 |     const isEnabled = await autoLaunch.isEnabled()
0031 |     if (!isEnabled && enable) {
0032 |         await autoLaunch.enable()
0033 |         return
0034 |     }
0035 |     if (isEnabled && !enable) {
0036 |         await autoLaunch.disable()
0037 |         return
0038 |     }
0039 | }
// ===== END FILE: src/main/autoLauncher.ts =====
// ===== FILE: src/main/cache.ts =====
0001 | export interface CacheItem<T> {

```

```

0002 |   value: T
0003 |   expireAt: number
0004 | }
0005 |
0006 | // In-memory cache store
0007 | const memoryCache = new Map<string, CacheItem<any>>()
0008 |
0009 | export async function cache<T>({
0010 |   key: string,
0011 |   getter: () => Promise<T>,
0012 |   options: {
0013 |     ttl: number // 缓存过期时间, 单位为毫秒
0014 |     refreshFallbackToCache?: boolean // 如果刷新时获取新值失败, 是否从缓存中继续使用过期的旧值
0015 |   }
0016 | }): Promise<T> {
0017 |   let cache = memoryCache.get(key) as CacheItem<T> | undefined
0018 |
0019 |   if (cache && cache.expireAt > Date.now()) {
0020 |     return cache.value
0021 |   }
0022 |
0023 |   try {
0024 |     const newValue = await getter()
0025 |     cache = {
0026 |       value: newValue,
0027 |       expireAt: Date.now() + options.ttl,
0028 |     }
0029 |     memoryCache.set(key, cache)
0030 |     return newValue
0031 |   } catch (e) {
0032 |     if (options.refreshFallbackToCache && cache) {
0033 |       return cache.value
0034 |     }
0035 |     throw e
0036 |   }
0037 | }
// ===== END FILE: src/main/cache.ts =====
// ===== FILE: src/main/deeplinks.ts =====
0001 | import type { BrowserWindow } from 'electron'
0002 |
0003 | export function handleDeepLink(mainWindow: BrowserWindow, link: string) {
0004 |   const normalizedLink = link.replace(/^aiclient-dev:\\\\/, 'aiclient://')
0005 |   const url = new URL(normalizedLink)
0006 |
0007 |   console.log('C Parsed URL:', { hostname: url.hostname, pathname: url.pathname,
0008 |
0009 |   // handle `aiclient://mcp/install?server=`
0010 |   if (url.hostname === 'mcp' && url.pathname === '/install') {
0011 |     const encodedConfig = url.searchParams.get('server') || ''
0012 |     mainWindow.webContents.send('navigate-to', `/settings/mcp?install=${encodeURIComponent`

```

```

0013 |   }
0014 | }
// ===== END FILE: src/main/deeplinks.ts =====
// ===== FILE: src/main/file-parser.ts =====
0001 | import * as chardet from 'chardet'
0002 | import Epub from 'epub'
0003 | import * as fs from 'fs-extra'
0004 | import * as iconv from 'iconv-lite'
0005 | import { isEpubFilePath, isOfficeFilePath } from '../shared/file-extensions'
0006 | import { getLogger } from './util'
0007 |
0008 | const log = getLogger('file-parser')
0009 |
0010 | // Helper function to decode HTML entities
0011 | function decodeHtmlEntities(text: string): string {
0012 |   // Handle hexadecimal entities like &#x6b64;
0013 |   text = text.replace(/&#x([0-9A-Fa-f]+)/g, (match, hex) => {
0014 |     try {
0015 |       return String.fromCharCode(parseInt(hex, 16))
0016 |     } catch (e) {
0017 |       return match // Return original if conversion fails
0018 |     }
0019 |   })
0020 |
0021 |   // Handle decimal entities like &#123;
0022 |   text = text.replace(/&#(\d+)/g, (match, dec) => {
0023 |     try {
0024 |       return String.fromCharCode(parseInt(dec, 10))
0025 |     } catch (e) {
0026 |       return match // Return original if conversion fails
0027 |     }
0028 |   })
0029 |
0030 |   // Handle named entities
0031 |   return text
0032 |     .replace(/&nbsp;/g, ' ')
0033 |     .replace(/&amp;/g, '&')
0034 |     .replace(/&lt;/g, '<')
0035 |     .replace(/&gt;/g, '>')
0036 |     .replace(/&quot;/g, '"')
0037 |     .replace(/&#39;/g, "'")
0038 |     .replace(/&apos;/g, "'")
0039 | }
0040 |
0041 | // Simple concurrent map implementation using native Promise.allSettled
0042 | async function concurrentMap<T, R>(
0043 |   items: T[],
0044 |   mapper: (item: T, index: number) => Promise<R>,
0045 |   concurrency: number = 8
0046 | ): Promise<R[]> {

```

```

0047 |   const results: R[] = []
0048 |
0049 |   for (let i = 0; i < items.length; i += concurrency) {
0050 |     const batch = items.slice(i, i + concurrency)
0051 |     const batchNumber = Math.floor(i / concurrency) + 1
0052 |     const totalBatches = Math.ceil(items.length / concurrency)
0053 |
function processFileBatch(batchNumber: number, totalBatches: number, subBatch: number, filePath: string) {
5 |   const batchResults = await Promise.allSettled(batch.map((item, batchIndex) =
6 |
7 |   // Extract successful results
8 |   for (const result of batchResults) {
9 |     if (result.status === 'fulfilled') {
0 |       results.push(result.value)
1 |     }
2 |   }
3 | }
4 | }
5 |
6 | return results
7 | }
8 |
9 | export async function parseFile(filePath: string) {
0 |   if (isOfficeFilePath(filePath)) {
1 |     try {
2 |       const officeParser = await import('officeparser')
3 |       const data = await officeParser.default.parseOfficeAsync(filePath)
4 |       return data
5 |     } catch (error) {
6 |       log.error(error)
7 |       throw error
8 |     }
9 |   }
0 |
1 |   if (isEpubFilePath(filePath)) {
2 |     try {
3 |       const data = await parseEpub(filePath)
4 |       return data
5 |     } catch (error) {
6 |       log.error(error)
7 |       throw error
8 |     }
9 |   }
0 |
1 |   // Read first 4KB for encoding detection to avoid memory issues with large fil
2 |   const stats = await fs.stat(filePath)
3 |   const sampleSize = Math.min(4096, stats.size)
4 |
5 |   // Read sample using createReadStream for partial file reading
6 |   const sampleBuffer = new Uint8Array(sampleSize)

```

```

0097 |   const fd = await fs.promises.open(filePath, 'r')
0098 |   await fd.read(sampleBuffer, 0, sampleSize, 0)
0099 |   await fd.close()
0100 |
0101 |   // Detect encoding from sample
0102 |   const detectedEncoding = chardet.detect(sampleBuffer)
0103 |   const encoding = detectedEncoding || 'utf8'
0104 |
0105 |   log.debug(`Detected encoding for ${filePath}: ${encoding}`)
0106 |
0107 |   // Read full file as buffer and convert with detected encoding
0108 |   const fileBuffer = await fs.readFile(filePath)
0109 |   const data = iconv.decode(fileBuffer, encoding)
0110 |   return data
0111 | }
0112 |
0113 | export async function parseEpub(filePath: string): Promise<string> {
0114 |   return new Promise((resolve, reject) => {
0115 |     const epub = new Epub(filePath)
0116 |
0117 |     epub.on('error', (error) => {
0118 |       log.error('EPUB parsing error:', error)
0119 |       reject(error)
0120 |     })
0121 |
0122 |     epub.on('end', async () => {
0123 |       try {
0124 |         const metadata = epub.metadata as { title?: string; creator?: string; la
0125 |         log.info('EPUB metadata:', {
0126 |           title: metadata.title,
0127 |           creator: metadata.creator,
0128 |           language: metadata.language,
0129 |           chapters: epub.flow.length,
0130 |         })
0131 |
0132 |         // Helper function to process a single chapter
0133 |         const processChapter = async (chapter: { id: string }): Promise<string |
0134 |         try {
0135 |           const chapterText = await new Promise<string>((resolveChapter, rejec
0136 |           epub.getChapter(chapter.id, (error, text) => {
0137 |             if (error) {
0138 |               log.error(`Error reading chapter ${chapter.id}:`, error)
0139 |               rejectChapter(error)
0140 |             } else {
0141 |               resolveChapter(text || '')
0142 |             }
0143 |           })
0144 |         })
0145 |
0146 |         // Remove HTML tags and extract plain text

```



```

0147 |         let plainText = chapterText.replace(/<[^>]*>/g, '') // Remove HTML t
0148 |
0149 |         // Decode HTML entities (including hex)
0150 |         plainText = decodeHtmlEntities(plainText)
0151 |         .replace(/\s+/g, ' ') // Replace multiple whitespaces with single
0152 |         .trim()
0153 |
0154 |         return plainText || null
0155 |     } catch (chapterError) {
0156 |         log.warn(`Failed to read chapter ${chapter.id}, skipping`, chapterE
0157 |         return null // Return null for failed chapters to continue processin
0158 |     }
0159 | }
0160 |
0161 | // Extract text from all chapters using concurrent processing
0162 | log.info(`Starting concurrent processing of ${epub.flow.length} chapters
0163 |
0164 | const chapterResults = await concurrentMap(epub.flow as { id: string }[])
0165 | const chapterTexts = chapterResults.filter((text: string | null) => text
0166 | log.info(`Successfully processed ${chapterTexts.length}/${epub.flow.leng
0167 |
0168 | const fullText = chapterTexts.join('\n\n')
0169 |
0170 | if (!fullText) {
0171 |     throw new Error('No readable text content found in EPUB file')
0172 | }
0173 |
0174 | log.info(`Successfully extracted ${fullText.length} characters from ${ch
0175 | resolve(fullText)
0176 | } catch (error) {
0177 |     log.error('Error extracting EPUB content:', error)
0178 |     reject(error)
0179 | }
0180 | })
0181 |
0182 | epub.parse()
0183 | })
0184 | }
// ===== END FILE: src/main/file-parser.ts =====
// ===== FILE: src/main/knowledge-base/db.ts =====
0001 | import fs from 'node:fs'
0002 | import path from 'node:path'
0003 | import type { Client } from '@libsql/client'
0004 | import { LibSQLVector } from '@mastra/libsql'
0005 | import { app } from 'electron'
0006 | import { sentry } from '../adapters/sentry'
0007 | import { getLogger } from '../util'
0008 |
0009 | const log = getLogger('knowledge-base:db')
0010 |

```

```

0011 | // Database file path
0012 | const dbDir = path.join(app.getPath('userData'), 'databases')
0013 | const primaryDbPath = path.join(dbDir, 'knowledge_base.db')
0014 | const legacyDbPath = path.join(dbDir, 'chatbox_kb.db')
0015 |
0016 | function resolveDbPath() {
0017 |   if (fs.existsSync(legacyDbPath) && !fs.existsSync(primaryDbPath)) {
0018 |     log.info('[DB] Using legacy database file path', { legacyDbPath })
0019 |     return legacyDbPath
0020 |   }
0021 |   return primaryDbPath
0022 | }
0023 |
0024 | const dbPath = resolveDbPath()
0025 |
0026 | // Ensure database directory exists
0027 | if (!fs.existsSync(dbDir)) {
0028 |   fs.mkdirSync(dbDir, { recursive: true })
0029 | }
0030 |
0031 | // Polyfill for mastra
0032 | if (typeof global.crypto === 'undefined' || !('subtle' in global.crypto)) {
0033 |   global.crypto = require('node:crypto')
0034 | }
0035 |
0036 | let db: Client
0037 | let vectorStore: LibSQLVector
0038 |
0039 | async function initDB(db: Client) {
0040 |   try {
0041 |     await db.batch([
0042 |       `CREATE TABLE IF NOT EXISTS knowledge_base (
0043 |         id INTEGER PRIMARY KEY AUTOINCREMENT,
0044 |         rate TEXT NOT NULL,
0045 |         embedding_model TEXT NOT NULL,
0046 |         rerank_model TEXT,
0047 |         vision_model TEXT,
0048 |         created_at DATETIME DEFAULT CURRENT_TIMESTAMP
0049 |       )`,
0050 |       `CREATE TABLE IF NOT EXISTS kb_file (
0051 |         id INTEGER PRIMARY KEY AUTOINCREMENT,
0052 |         kb_id INTEGER NOT NULL,
0053 |         filename TEXT NOT NULL,
0054 |         filepath TEXT NOT NULL,
0055 |         mime_type TEXT NOT NULL,
0056 |         file_size INTEGER DEFAULT 0,
0057 |         chunk_count INTEGER DEFAULT 0,
0058 |         total_chunks INTEGER DEFAULT 0,
0059 |         status TEXT NOT NULL DEFAULT 'pending',
0060 |         error TEXT,

```

0061	1	created at	DATETIME	CURRENT_TIMESTAMP	0062	III	process doing operations at CURRENT_TIMESTAMP
					0063		FOREIGN KEY (ss_id) REFERENCES knowledge_base(ss_id)
					0064		};
					0065		};
adding databases)					0066		// Add total_chunks column if it doesn't exist (for exi
chunks THROUGH TRIGGER					0067		wait db.batch(['ALTER TABLE kg_file ADD COLUMN total_c
if 'skip' exists column no					0068		if (error instanceof Error && error.message.includes
error).					0069		log.error(['DB'] Failed to add total_chunks column
					0070		) else {
					0071		// Ignore error if column already exists
column already exists).					0072		log.info(['DB'] Database initialized (total_chunks c
					0073		);
					0074		});
					0075		
JSON format. NOT. use					0076		// Add document_parser column to knowledge_base table
as ADD COLUMN document_parser TEXT					0077		wait db.batch(['ALTER TABLE knowledge_ba
document_parser('skip' exists column no					0078		if (error instanceof Error && error.message
document_parser', error)					0079		log.error(['DB'] Failed to add document
					0080		);
					0081		});
					0082		
to record which parser was used.					0083		// Add parser_type column to kg_file table
COLUMN parser_type TEXT THROUGH 'to					0084		wait db.batch(['ALTER TABLE kg_file ADD
message.includes('skip' exists column no					0085		if (error instanceof Error && error.me
types column', error)					0086		log.error(['DB'] Failed to add parser
					0087		);
					0088		});
					0089		
					0090		log.info(['DB'] Database initialized);
					0091		catch (error) {
error', error)					0092		log.error(['DB'] Failed to initialize data
					0093		
					0094		entry.withScope((scope) => {
ss=db').					0095		scope.set('component', 'knowledge-base
initialization');					0096		scope.set('operation', 'database_ini
					0097		scope.set('dbPath', dbPath);
					0098		entry.catchException(error)
					0099		});
					0100		throw error
					0101		};
					0102		};
					0103		
					0104		export async function initializeDatabase() {
					0105		try {
					0106		const mongo = new MongoClient({
					0107		connectionUrl: `mongodb://localhost:27017/\${dbPath}`,
					0108		});
时操作一个 db 文件会导致数据库损坏					0109		// 这里不再创建新的 client, 因为多个 client 同
throws because internal property					0110		// ignore ignore limit/operation/operation

```

0111 |     db = (vectorStore as any).turso
0112 |     await initDB(db)
0113 |
0114 |     // Clean up any processing files left from previous session
0115 |     await cleanupProcessingFiles()
0116 |   } catch (error) {
0117 |     log.error('[DB] Failed to initialize database system:', error)
0118 |     sentry.withScope((scope) => {
0119 |       scope.setTag('component', 'knowledge-base-db')
0120 |       scope.setTag('operation', 'vector_store_initialization')
0121 |       scope.setExtra('dbPath', dbPath)
0122 |       sentry.captureException(error)
0123 |     })
0124 |     throw error
0125 |   }
0126 | }
0127 |
0128 | export function getDatabase(): Client {
0129 |   if (!db) {
0130 |     const error = new Error('Database not initialized')
0131 |     log.error('[DB] Database not initialized')
0132 |     sentry.withScope((scope) => {
0133 |       scope.setTag('component', 'knowledge-base-db')
0134 |       scope.setTag('operation', 'database_access')
0135 |       sentry.captureException(error)
0136 |     })
0137 |     throw error
0138 |   }
0139 |   return db
0140 | }
0141 |
0142 | export function getVectorStore(): LibSQLVector {
0143 |   if (!vectorStore) {
0144 |     const error = new Error('Vector store not initialized')
0145 |     log.error('[DB] Vector store not initialized')
0146 |     sentry.withScope((scope) => {
0147 |       scope.setTag('component', 'knowledge-base-db')
0148 |       scope.setTag('operation', 'vector_store_access')
0149 |       sentry.captureException(error)
0150 |     })
0151 |     throw error
0152 |   }
0153 |   return vectorStore
0154 | }
0155 |
0156 | // Helper function to parse SQLite timestamp correctly
0157 | export function parseSQLiteTimestamp(sqliteTimestamp: string): number {
0158 |   try {
0159 |     // SQLite CURRENT_TIMESTAMP returns UTC time in format: 'YYYY-MM-DD HH:MM:SS'
0160 |     // We need to explicitly tell JavaScript this is UTC time

```



```

0161 |     const utcDate = new Date(`${sqliteTimestamp} UTC`)
0162 |     const timestamp = utcDate.getTime()
0163 |
0164 |     if (Number.isNaN(timestamp)) {
0165 |         throw new Error(`Invalid timestamp format: ${sqliteTimestamp}`)
0166 |     }
0167 |
0168 |     return timestamp
0169 | } catch (error) {
0170 |     log.error(`[DB] Failed to parse SQLite timestamp: ${sqliteTimestamp}`, error)
0171 | }
0172 | withTransaction(async (scope) => {
0173 |     scope.setTag('component', 'knowledge-base-db')
0174 |     scope.setTag('operation', 'timestamp_parsing')
0175 |     scope.setExtra('sqliteTimestamp', sqliteTimestamp)
0176 |     sentry.captureException(error)
0177 | })
0178 | // Return current timestamp as fallback
0179 | return Date.now()
0180 | }
0181 |
0182 | // Transaction wrapper - ensures atomicity of database operations
0183 | export async function withTransaction<T>(operation: () => Promise<T>): Promise<T> {
0184 |     const db = getDatabase()
0185 |     const transactionId = Math.random().toString(36).slice(2, 10)
0186 |
0187 |     try {
0188 |         log.debug(`[DB] Starting transaction ${transactionId}`)
0189 |         await db.execute('BEGIN TRANSACTION')
0190 |         const result = await operation()
0191 |         await db.execute('COMMIT')
0192 |         log.debug(`[DB] Transaction ${transactionId} committed successfully`)
0193 |         return result
0194 |     } catch (error) {
0195 |         log.error(`[DB] Transaction ${transactionId} failed:`, error)
0196 |
0197 |         try {
0198 |             await db.execute('ROLLBACK')
0199 |             log.debug(`[DB] Transaction ${transactionId} rolled back`)
0200 |         } catch (rollbackError) {
0201 |             log.error(`[DB] Failed to rollback transaction ${transactionId}:`, rollbackError)
0202 |             sentry.withScope((scope) => {
0203 |                 scope.setTag('component', 'knowledge-base-db')
0204 |                 scope.setTag('operation', 'transaction_rollback')
0205 |                 scope.setExtra('transactionId', transactionId)
0206 |                 sentry.captureException(rollbackError)
0207 |             })
0208 |         }
0209 |
0210 |         // Report transaction failures to Sentry for critical operations

```



```

0211 |     sentry.withScope((scope) => {
0212 |         scope.setTag('component', 'knowledge-base-db')
0213 |         scope.setTag('operation', 'transaction_failure')
0214 |         scope.setExtra('transactionId', transactionId)
0215 |         sentry.captureException(error)
0216 |     })
0217 |
0218 |     throw error
0219 | }
0220 | }
0221 |
0222 | // Cleanup processing files that may have been left from previous session
0223 | async function cleanupProcessingFiles() {
0224 |     try {
0225 |         log.debug('[DB] Cleaning up processing files from previous session...')
0226 |         const result = await db.execute({
0227 |             sql: 'UPDATE kb_file SET status = ?, processing_started_at = NULL WHERE st
0228 |             args: ['paused', 'processing'],
0229 |         })
0230 |         const affectedRows = result.rowsAffected || 0
0231 |         if (affectedRows > 0) {
0232 |             log.debug(`[DB] Set ${affectedRows} interrupted processing files to paused
0233 |         }
0234 |     } catch (err) {
0235 |         log.error('[DB] Failed to cleanup processing files:', err)
0236 |         sentry.withScope((scope) => {
0237 |             scope.setTag('component', 'knowledge-base-db')
0238 |             scope.setTag('operation', 'cleanup_processing_files')
0239 |             sentry.captureException(err)
0240 |         })
0241 |     }
0242 | }
0243 |
0244 | // Check for timed out processing files and mark them as failed
0245 | export async function checkProcessingTimeouts() {
0246 |     try {
0247 |         // Files, processing for more than 5 minutes should be marked as failed
0248 |         const timeoutMinutes = 5
0249 |         const timeoutThreshold = new Date(Date.now() - timeoutMinutes * 60 * 1000).t
0250 |
0251 |         const db = getDatabase()
0252 |
0253 |         // Find processing files that started before the timeout threshold
0254 |         const rs = await db.execute({
0255 |             sql: `SELECT id, filename FROM kb_file
0256 |                 WHERE status = 'processing'
0257 |                 AND processing_started_at IS NOT NULL
0258 |                 AND datetime(processing_started_at) < datetime(?)`,
0259 |             args: [timeoutThreshold],
0260 |         })

```

```

0261 |
0262 |     if (rs.rows.length > 0) {
0263 |         log.debug(`[DB] Found ${rs.rows.length} timed out processing files`)
0264 |
0265 |         // Mark them as failed
0266 |         for (const file of rs.rows) {
0267 |             await db.execute(//
sql: `UPDATE db-file SET status = 2, error = 2, processing_started_at`
0269 |             args: ['failed', 'Processing timeout after ${timeoutMinutes} minutes`
0270 |             )
0271 |             log.debug(`[DB] Marked file as failed due to timeout: ${file.filename}`)
0272 |         }
0273 |     }
0274 | } catch (err) {
0275 |     log.error('[DB] Failed to check processing timeouts:', err)
0276 | }
0277 | }
// ===== END FILE: src/main/knowledge-base/db.ts =====
// ===== FILE: src/main/knowledge-base/file-loaders.ts =====
0001 | import { setTimeout } from 'node:timers/promises'
0002 | import { MDocument } from '@mastra/rag'
0003 | import { embedMany } from 'ai'
0004 | import type { DocumentParserConfig } from '../../../shared/types/settings'
0005 | import { rerank } from '../../../shared/models/rerank'
0006 | import { sentry } from '../adapters/sentry'
0007 | import { getLogger } from 'logtail'
0008 | import { checkProcessingTimeouts, getDatabase, getVectorStore } from './
0009 | import { getEmbeddingProvider, getRerankProvider } from './model-provid
0010 | import { getEffectiveParserConfig, parseFileWithRouter, type ParserFile
0011 |
0012 | const log = getLogger('knowledge-base:file-loaders')
0013 |
0014 | /**
0015 |  * Parse error message to extract user-friendly message
0016 |  * Handles JSON error responses from parser providers
0017 |  */
0018 | function parseErrorMessage(errorMessage: string): string {
0019 |     try {
0020 |         const jsonMatch = errorMessage.match(/(?:[{}\\"]|\\\\|\\")+/)
0021 |         if (jsonMatch) {
0022 |             const jsonStr = jsonMatch[0]
0023 |             const parsed = JSON.parse(jsonStr)
0024 |             if (parsed.error?.detail) {
0025 |                 return parsed.error.detail
0026 |             }
0027 |             if (parsed.error?.title) {
0028 |                 return parsed.error.title
0029 |             }
0030 |         }
0031 |     } catch {}

```

```

0032 |     // JSON parsing failed, return original message
0033 | }
0034 | return errorMessage
0035 | }
0036 |
0037 | // Parse file to MDocument using the parser router
0038 | async function parseFileToDocumentWithRouter(
0039 |   filePath: string,
0040 |   fileMeta: ParserFileMeta,
0041 |   kbId: number,
0042 |   parserConfig: DocumentParserConfig
0043 | ): Promise<{ document: MDocument; parserUsed: string }> {
0044 |   log.info(`[FILE] Parsing ${fileMeta.filename} with ${parserConfig.type} parser
0045 |
0046 |   const result = await parseFileWithRouter(filePath, fileMeta, parserConfig, kbId
0047 |
0048 |   log.info(`[FILE] Parse completed for ${fileMeta.filename}, parser used: ${resu
0049 |
0050 |   // Convert content to MDocument based on content type
0051 |   const document = MDocument.fromText(result.content)
0052 |   return { document, parserUsed: result.parserUsed }
0053 | }
0054 |
0055 | // Use mastra to parse, chunk, embed, and store files
0056 | export async function processFileWithMastra(
0057 |   filePath: string,
0058 |   fileMeta: { fileId: number; filename: string; mimeType: string },
0059 |   kbId: number,
0060 |   parserConfig: DocumentParserConfig
0061 | ) {
0062 |   const startTime = Date.now()
0063 |   log.debug(
0064 |     `[FILE] Starting file processing: ${fileMeta.filename} (id=${fileMeta.fileId
0065 |   )
0066 |
0067 |   try {
0068 |     const db = getDatabase()
0069 |
0070 |     // Check current processing status and get processed chunk count
0071 |     const fileRecord = await db.execute('SELECT chunk_count, total_chunks, statu
0072 |       fileMeta.fileId,
0073 |     ])
0074 |     const currentChunkCount = (fileRecord.rows[0]?.chunk_count as number) || 0
0075 |     const currentTotalChunks = (fileRecord.rows[0]?.total_chunks as number) || 0
0076 |
0077 |     // 1. Parse file using the parser router
0078 |     const parseResult = await parseFileToDocumentWithRouter(filePath, fileMeta,
0079 |     const doc = parseResult.document
0080 |     const parserUsed = parseResult.parserUsed
0081 |

```

```

0082 | // Update parser_type in database
0083 | await db.execute({
0084 |   sql: 'UPDATE kb_file SET parser_type = ? WHERE id = ?',
0085 |   args: [parserUsed, fileMeta.fileId],
0086 | })
0087 |
0088 | // 2. Chunking
0089 | const allChunks = await doc.chunk({
0090 |   strategy: 'recursive',
0091 |   size: 512,
0092 |   overlap: 50,
0093 | })
0094 |
0095 | if (!allChunks || allChunks.length === 0) {
0096 |   // Third-party cloud parsing may return empty content for truly empty file
0097 |   if (parserUsed === 'mineru') {
0098 |     await db.execute({
0099 |       sql: 'UPDATE kb_file SET chunk_count = 0, status = ? WHERE id = ?',
0100 |       args: ['done', fileMeta.fileId],
0101 |     })
0102 |   } else {
0103 |     throw new Error('No content extracted from file')
0104 |   }
0105 |   return
0106 | }
0107 |
0108 | // Record total chunks if not already recorded
0109 | if (currentTotalChunks === 0 || currentTotalChunks !== allChunks.length) {
0110 |   await db.execute({
0111 |     sql: 'UPDATE kb_file SET total_chunks = ? WHERE id = ?',
0112 |     args: [allChunks.length, fileMeta.fileId],
0113 |   })
0114 |   log.debug('[FILE] Recorded total chunks: ${allChunks.length} for file ${fi
0115 | }
0116 |
0117 | log.debug('[FILE] Processing progress: ${currentChunkCount}/${allChunks.leng
0118 |
0119 | // 3. Check if processing is already complete
0120 | if (currentChunkCount >= allChunks.length) {
0121 |   log.info('[FILE] File already fully processed: ${fileMeta.filename} (id=${
0122 |   return
0123 | }
0124 |
0125 | // 4. Get remaining chunks to process
0126 | const remainingChunks = allChunks.slice(currentChunkCount)
0127 | log.debug('[FILE] Processing remaining ${remainingChunks.length} chunks from
0128 |
0129 | // 5. If no remaining chunks, processing is complete
0130 | if (remainingChunks.length === 0) {
0131 |   log.info('[FILE] File processing already complete: ${fileMeta.filename} (i

```

```

0132 |         return
0133 |     }
0134 |
0135 |     // 6. Process remaining chunks in batches
0136 |     const embeddingInstance = await getEmbeddingProvider(kbId)
0137 |     const vectorStore = getVectorStore()
0138 |     const indexName = `kb_${kbId}`
0139 |     const BATCH_SIZE = 50 // Process chunks in batches of 50
0140 |
0141 |     // Ensure vector index exists by getting dimension from first remaining chunk
0142 |     const firstEmbedding = await embedMany({
0143 |         model: embeddingInstance,
0144 |         values: [`filename: ${fileMeta.filename}\nchunk:\n${remainingChunks[0].text}`]
0145 |     })
0146 |     await vectorStore.createIndex({ indexName, dimension: firstEmbedding.embeddingDimension })
0147 |
0148 |     for (let i = 0; i < remainingChunks.length; i += BATCH_SIZE) {
0149 |         // Check if file has been paused before processing each batch
0150 |         const statusCheck = await db.execute('SELECT status FROM kb_file WHERE id = ?')
0151 |         const currentStatus = statusCheck.rows[0]?.status as string
0152 |         if (currentStatus === 'paused') {
0153 |             log.info('[FILE] File processing paused by user: ${fileMeta.filename}')
0154 |             return
0155 |         }
0156 |
0157 |         const batchChunks = remainingChunks.slice(i, i + BATCH_SIZE)
0158 |         const batchTexts = batchChunks.map((chunk: any) => `filename: ${fileMeta.filename}\nchunk:\n${chunk.text}`)
0159 |
0160 |         const batchNumber = Math.floor(i / BATCH_SIZE) + 1
0161 |         const totalBatches = Math.ceil(remainingChunks.length / BATCH_SIZE)
0162 |         log.debug('[FILE] Processing batch ${batchNumber}/${totalBatches}, chunks: ${batchTexts.length}')
0163 |
0164 |         // Generate embeddings for this batch
0165 |         const embeddingResult = await embedMany({
0166 |             model: embeddingInstance,
0167 |             values: batchTexts,
0168 |         })
0169 |
0170 |         if (!embeddingResult.embeddings || embeddingResult.embeddings.length !== batchTexts.length) {
0171 |             throw new Error(`Embedding batch failed: expected ${batchTexts.length}, got ${embeddingResult.embeddings.length}`)
0172 |         }
0173 |
0174 |         // Store vectors for this batch
0175 |         log.debug('[FILE] Storing batch ${batchNumber}/${totalBatches} to vector store')
0176 |         await vectorStore.upsert({
0177 |             indexName,
0178 |             vectors: embeddingResult.embeddings,
0179 |             metadata: batchChunks.map((chunk: any, chunkIndex: number) => ({
0180 |                 filename: fileMeta.filename,
0181 |                 chunkIndex,
0182 |                 text: chunk.text,
0183 |             })),
0184 |         })
0185 |     }

```



```

0182 |         text: chunk.text,
0183 |         fileId: fileMeta.fileId,
0184 |         filename: fileMeta.filename,
0185 |         mimeType: fileMeta.mimeType,
0186 |         chunkIndex: currentChunkCount + i + chunkIndex, // Use absolute chunk
0187 |     })),
0188 | })
0189 |
0190 | // Update processed chunk count in database
0191 | const newChunkCount = currentChunkCount + i + batchChunks.length
0192 | await db.execute({
0193 |     sql: 'UPDATE kb_file SET chunk_count = ? WHERE id = ?',
0194 |     args: [newChunkCount, fileMeta.fileId],
0195 | })
0196 |
0197 | log.debug('[FILE] Updated chunk count to ${newChunkCount} for file ${fileM
0198 |
0199 | // Small delay between batches to avoid overwhelming the API
0200 | if (i + BATCH_SIZE < remainingChunks.length) {
0201 |     await setTimeout(100) // 100ms delay between batches
0202 | }
0203 | }
0204 |
0205 | const duration = Date.now() - startTime
0206 | log.info(
0207 |     '[FILE] File processed successfully: ${fileMeta.filename} (id=${fileMeta.f
0208 | )
0209 | // Mark as done and clear processing timestamp
0210 | await db.execute({
0211 |     sql: 'UPDATE kb_file SET status = ?, processing_started_at = NULL WHERE id
0212 |     args: ['done', fileMeta.fileId],
0213 | })
0214 | } catch (error: any) {
0215 |     const duration = Date.now() - startTime
0216 |     log.error('[FILE] File processing failed after ${duration}ms: ${fileMeta.fil
0217 |
0218 | // Determine the operation type based on error message for better debugging
0219 | let operation = 'file_processing'
0220 | if (error.message.includes('parse')) {
0221 |     operation = 'file_parsing'
0222 | } else if (error.message.includes('chunk')) {
0223 |     operation = 'document_chunking'
0224 | } else if (error.message.includes('embedding')) {
0225 |     operation = 'generate_embeddings'
0226 | } else if (error.message.includes('store') || error.message.includes('vector
0227 |     operation = 'vector_storage'
0228 | } else if (error.message.includes('vision') || error.message.includes('OCR'))
0229 |     operation = 'image_ocr_processing'
0230 | }
0231 |

```

```

0232 | // Report processing failures to Sentry with unified context
0233 | sentry.withScope((scope) => {
0234 |   scope.setTag('component', 'knowledge-base-file')
0235 |   scope.setTag('operation', operation)
0236 |   scope.setExtra('fileId', fileMeta.fileId)
0237 |   scope.setExtra('filename', fileMeta.filename)
0238 |   scope.setExtra('mimeType', fileMeta.mimeType)
0239 |   scope.setExtra('kbId', kbId)
0240 |   scope.setExtra('duration', duration)
0241 |   scope.setExtra('filePath', filePath)
0242 |   sentry.captureMessage('Error')
0243 | })
0244 |
0245 |   throw error
0246 | }
0247 | }
0248 |
0249 | async function processPendingFiles() {
0250 |   try {
0251 |     // First check for timed out processing files.
0252 |     await checkProcessingTimeouts()
0253 |
0254 |     const db = getDatabase()
0255 |     // Query pending files with their KB's parent config
0256 |     const rs = await db.execute(
0257 |       `
0258 |       SELECT F.F, Hs.document_parent as Hs_document_parent
0259 |       FROM Hs_file F
0260 |       JOIN knowledge_base Kb ON F.kb_id = Kb.id
0261 |       WHERE F.status = ?
0262 |       `,
0263 |       ['pending']
0264 |     )
0265 |
0266 |     if (rs.rows.length === 0) {
0267 |       return
0268 |     }
0269 |
0270 |     log.debug(`[FTR] Processing ${rs.rows.length} pending files`)
0271 |
0272 |     for (const file of rs.rows) {
0273 |       // Parse KB parent config
0274 |       let kbParentConfig: DocumentParentConfig | undefined
0275 |       if (file.kb_document_parent) {
0276 |         try {
0277 |           kbParentConfig = JSON.parse(file.kb_document_parent as string)
0278 |         } catch {
0279 |           log.warn(`[FTR] Failed to parse KB document_parent config for file ${
0280 |             file.F
0281 |           }`)
0282 |         }
0283 |       }
0284 |     }
0285 |   }

```

```

0282 |
0283 |     const effectiveParserConfig: DocumentParserConfig = getEffectiveParserConf
0284 |
0285 |     try {
0286 |         log.debug(
0287 |             `[FILE] Processing file: ${file.filename} (id=${file.id}, parser=${eff
0288 |         )
0289 |
0290 |         // Mark as processing, record the processing start time, and save parser
0291 |         // We set parser_type here at the start so that if parsing fails, the er
0292 |         await db.execute({
0293 |             sql: `UPDATE kb_file SET status = ?, processing_started_at = CURRENT_T
0294 |             args: ['processing', effectiveParserConfig.type, file.id],
0295 |         })
0296 |
0297 |         // Use mastra to parse, chunk, embed, and store (supports resuming from
0298 |         await processFileWithMastra(
0299 |             file.filepath as string,
0300 |             { fileId: file.id as number, filename: file.filename as string, mimeTy
0301 |             file.kb_id as number,
0302 |             effectiveParserConfig
0303 |         )
0304 |     } catch (err: any) {
0305 |         log.error(`[FILE] File processing failed: ${file.filename} (id=${file.id
0306 |         // Mark as failed - parse error message to extract user-friendly message
0307 |         const rawErrorMessage = err instanceof Error ? err.message : String(err)
0308 |         const errorMessage = parseErrorMessage(rawErrorMessage)
0309 |         await db.execute({
0310 |             sql: `UPDATE kb_file SET status = ?, error = ?, processing_started_at
0311 |             args: ['failed', errorMessage, file.id],
0312 |         })
0313 |
0314 |         // Report individual file processing failures
0315 |         sentry.withScope((scope) => {
0316 |             scope.setTag('component', 'knowledge-base-file')
0317 |             scope.setTag('operation', 'individual_file_processing')
0318 |             scope.setExtra('fileId', file.id)
0319 |             scope.setExtra('filename', file.filename)
0320 |             scope.setExtra('kbId', file.kb_id)
0321 |             scope.setExtra('parserType', effectiveParserConfig.type)
0322 |             sentry.captureException(err)
0323 |         })
0324 |     }
0325 | }
0326 | } catch (error: any) {
0327 |     log.error('[FILE] Failed to process pending files:', error)
0328 |     sentry.withScope((scope) => {
0329 |         scope.setTag('component', 'knowledge-base-file')
0330 |         scope.setTag('operation', 'process_pending_files')
0331 |         sentry.captureException(error)

```

```

0332 |     })
0333 |   }
0334 | }
0335 |
0336 | // Periodic polling
0337 | export async function startWorkerLoop() {
0338 |   log.info('[FILE] Starting worker loop')
0339 |
0340 |   while (true) {
0341 |     try {
0342 |       await processPendingFiles()
0343 |     } catch (e: any) {
0344 |       log.error('[FILE] Worker loop error:', e)
0345 |       sentry.withScope((scope) => {
0346 |         scope.setTag('component', 'knowledge-base-file')
0347 |         scope.setTag('operation', 'worker_loop')
0348 |         sentry.captureException(e)
0349 |       })
0350 |
0351 |       // Wait before retrying to prevent rapid error loops
0352 |       await setTimeout(10000) // 10 seconds
0353 |     }
0354 |     await setTimeout(3000) // Poll every 3 seconds
0355 |   }
0356 | }
0357 |
0358 | // Search interface, embeddingProvider parameter is required
0359 | export async function searchKnowledgeBase(kbId: number, query: string) {
0360 |   try {
0361 |     log.debug(`[FILE] Searching knowledge base: kbId=${kbId}, query=${query}`)
0362 |     const embeddingInstance = await getEmbeddingProvider(kbId)
0363 |     const embedding = await embedMany({
0364 |       model: embeddingInstance,
0365 |       values: [query],
0366 |     })
0367 |     const vectorStore = getVectorStore()
0368 |     const indexName = `kb_${kbId}`
0369 |     const results = await vectorStore.query({
0370 |       indexName,
0371 |       queryVector: embedding.embeddings[0],
0372 |       topK: 20,
0373 |     })
0374 |     try {
0375 |       const rerankInstance = await getRerankProvider(kbId)
0376 |       if (rerankInstance) {
0377 |         const rerankedResults = await rerank(results, query, rerankInstance, {
0378 |           topK: 5,
0379 |         })
0380 |         return rerankedResults.map((r) => ({
0381 |           id: r.result.id,

```

```

0382 |         score: r.result.score,
0383 |         ...r.result.metadata,
0384 |     )))
0385 |     }
0386 |     return results.map((r) => ({
0387 |         id: r.id,
0388 |         score: r.score,
0389 |         ...r.metadata,
0390 |     }))
0391 | } catch (e) {
0392 |   log.error(`[FILE] Failed to rerank: kbId=${kbId}, query=${query}`, e)
0393 |   sentry.withScope((scope) => {
0394 |     scope.setTag('component', 'knowledge-base-file')
0395 |     scope.setTag('operation', 'rerank')
0396 |     scope.setExtra('kbId', kbId)
0397 |     scope.setExtra('query', query)
0398 |     sentry.captureException(e)
0399 |   })
0400 |   return results.map((r) => ({
0401 |     id: r.id,
0402 |     score: r.score,
0403 |     ...r.metadata,
0404 |   }))
0405 | }
0406 | } catch (e) {
0407 |   log.error(`[FILE] Failed to search: kbId=${kbId}, query=${query}`, e)
0408 |
0409 |   sentry.withScope((scope) => {
0410 |     scope.setTag('component', 'knowledge-base-file')
0411 |     scope.setTag('operation', 'search_knowledge_base')
0412 |     scope.setExtra('kbId', kbId)
0413 |     scope.setExtra('query', query)
0414 |     sentry.captureException(e)
0415 |   })
0416 | }
0417 | // TODO: user friendly error message

```

```

0418 |   throw e
0419 | }
0420 | }

```

```

) fileId: number, chunkId:
chunk=${chunks.length}) {

```

```

0421 | // Read chunks from vector store.
0422 | export async function readChunks(kbId: number, chunks:
0423 | for (
0424 |   log.debug(`[FILE] Reading chunks: kbId=${kbId}, ch
0425 |
0426 |
0427 |   if (!chunks || chunks.length === 0) {
0428 |     return []
0429 |   }
0430 |
0431 |   const indexName = `kb_${kbId}`

```



```

0432 |     const results: any[] = []
0433 |
0434 |     // Use single SQL query to get all chunks at once
0435 |     log.debug(`[FILE] Using single SQL query via vectorStore.turso for ${chunks.
0436 |
0437 |     const vectorStore = getVectorStore()
0438 |     // Build composite IN condition to avoid SQLite's 999 variable limit
0439 |     const valuePlaceholders = chunks.map(() => '(?,?)').join(',')
0440 |     const condition = `(json_extract(metadata, '$.fileId'), json_extract(metadata
0441 |
0442 |     // Flatten chunk parameters for the query
0443 |     const args = chunks.flatMap(chunk => chunk.fileId ? [chunk.fileId, chunk.index] : []);
0444 |
0445 |     const sql = `SELECT metadata FROM ${indexName} WHERE ${condition}`
0446 |     log.debug(`[FILE] Executing SQL: ${sql}`)
0447 |     log.debug(`[FILE] With args:`, args)
0448 |
0449 |     const queryResult = await (vectorStore as any).turso.execute({
0450 |       sql,
0451 |       args,
0452 |     })
0453 |
0454 |     log.debug(`[FILE] Single SQL query returned ${queryResult.rows.length} results`)
0455 |
0456 |     // Process results and maintain the order of the requested chunks
0457 |     const foundChunks = queryResult.rows.map((row: any) => {
0458 |       const metadata = JSON.parse(row.metadata as string)
0459 |       return {
0460 |         fileId: metadata.fileId,
0461 |         filename: metadata.filename,
0462 |         chunkIndex: metadata.chunkIndex,
0463 |         text: metadata.text,
0464 |       }
0465 |     })
0466 |
0467 |     // Maintain the order of the requested chunks
0468 |     for (const chunk of chunks) {
0469 |       const found = foundChunks.find(
0470 |         (fc: any) => Number(fc.fileId) === Number(chunk.fileId) && Number(fc.chunkIndex) === Number(chunk.chunkIndex)
0471 |       )
0472 |       if (found) {
0473 |         results.push(found)
0474 |       }
0475 |     }
0476 |
0477 |     return results
0478 |   } catch (sqlErr: any) {
0479 |     log.error(`[FILE] Single SQL query failed:`, sqlErr)
0480 |     sentry.withScope((scope) => {
0481 |       scope.setTag('component', 'knowledge-base-file')

```

```

0482 |         scope.setTag('operation', 'read_chunks')
0483 |         scope.setExtra('kbId', kbId)
0484 |         scope.setExtra('chunkCount', chunks.length)
0485 |         sentry.captureException(sqlErr)
0486 |     })
0487 |     throw sqlErr
0488 | }
0489 | }
// ===== END FILE: src/main/knowledge-base/file-loaders.ts =====
// ===== FILE: src/main/knowledge-base/index.ts =====
0001 | import { sentry } from '../adapters/sentry'
0002 | import { getLogger } from '../util'
0003 | import { initializeDatabase } from './db'
0004 | import { startWorkerLoop } from './file-loaders'
0005 | import { registerKnowledgeBaseHandlers } from './ipc-handlers'
0006 |
0007 | const log = getLogger('knowledge-base:index')
0008 |
0009 | let initPromise: Promise<void> | null = null
0010 |
0011 | async function initializeKnowledgeBase() {
0012 |     const startTime = Date.now()
0013 |     log.info('[KB] Initializing knowledge base system...')
0014 |
0015 |     try {
0016 |         // Register IPC handlers
0017 |         registerKnowledgeBaseHandlers()
0018 |         log.debug('[KB] IPC handlers registered')
0019 |
0020 |         // Initialize database and vector store
0021 |         await initializeDatabase()
0022 |         log.debug('[KB] Database initialized')
0023 |
0024 |         // Start background file processing worker
0025 |         startWorkerLoop()
0026 |         log.debug('[KB] Worker loop started')
0027 |
0028 |         const duration = Date.now() - startTime
0029 |         log.info(`[KB] Knowledge base system initialized successfully in ${duration}`)
0030 |     } catch (error) {
0031 |         const duration = Date.now() - startTime
0032 |         log.error(`[KB] Failed to initialize knowledge base system after ${duration}`)
0033 |
0034 |         // Report critical initialization errors to Sentry
0035 |         sentry.withScope((scope) => {
0036 |             scope.setTag('component', 'knowledge-base')
0037 |             scope.setTag('operation', 'initialization')
0038 |             scope.setExtra('duration', duration)
0039 |             scope.setExtra('error_type', 'initialization_failure')
0040 |             sentry.captureException(error)

```

```

0041 |     })
0042 |
0043 |     throw error
0044 |   }
0045 | }
0046 |
0047 | export function getInitPromise() {
0048 |   if (!initPromise) {
0049 |     initPromise = initializeKnowledgeBase()
0050 |   }
0051 |   return initPromise
0052 | }
0053 |
0054 | 0054 | // Don't throw error here while module is loaded with error-handling
0055 | getInitPromise().catch((error) => {
0056 |   log.error('[KB] Knowledge base auto-initialization failed:', error)
0057 |   // Don't rethrow here to avoid unhandled promise rejection
0058 | })
0059 |
0060 | // Re-export public APIs for external use
0061 | export { getDatabase, getVectorStore, parseSQLiteTimestamp, withTransaction } fr
0062 | export { readChunks, searchKnowledgeBase } from './file-loaders'
0063 | export { getEmbeddingProvider, getRerankProvider, getVisionProvider } from './mo
// ===== END FILE: src/main/knowledge-base/index.ts =====
// ===== FILE: src/main/knowledge-base/ipc-handlers.ts =====
0001 | import { ipcMain } from 'electron'
0002 | import type { FileMeta } from 'src/shared/types'
0003 | import { sentry } from '../adapters/sentry'
0004 | import { getLogger } from '../util'
0005 | import { getDatabase, getVectorStore, parseSQLiteTimestamp, withTransaction } fr
0006 | import { readChunks, searchKnowledgeBase } from './file-loaders'
0007 | import { MineruParser, testMineruConnection } from './parsers'
0008 |
0009 | const log = getLogger('knowledge-base:ipc-handlers')
0010 |
0011 | // Store active MinerU parsing tasks for cancellation support
0012 | // Key: filePath, Value: AbortController
0013 | const activeMineruParseTasks = new Map<string, AbortController>()
0014 |
0015 | // Register knowledge base related APIs
0016 | export function registerKnowledgeBaseHandlers() {
0017 |   // Knowledge Base CRUD operations
0018 |   ipcMain.handle('kb:list', async () => {
0019 |     try {
0020 |       log.debug('ipcMain: kb:list')
0021 |       const db = getDatabase()
0022 |       const rs = await db.execute('SELECT * FROM knowledge_base')
0023 |       return rs.rows.map((row) => ({
0024 |         id: row.id,
0025 |         name: row.name,

```

```

0026 |         embeddingModel: row.embedding_model,
0027 |         rerankModel: row.rerank_model,
0028 |         visionModel: row.vision_model,
0029 |         documentParser: row.document_parser ? JSON.parse(row.document_parser as
0030 |         createdAt: row.created_at,
0031 |     )))
0032 |   } catch (error: any) {
0033 |     log.error('ipcMain: kb:list failed', error)
0034 |     sentry.withScope((scope) => {
0035 |       scope.setTag('component', 'knowledge-base-ipc')
0036 |       scope.setTag('operation', 'kb_list')
0037 |       sentry.captureException(error)
0038 |     })
0039 |     throw error
0040 |   }
0041 | })
0042 |
0043 | ipcMain.handle(
0044 |   'kb:create',
0045 |   async (
0046 |     _event,
0047 |     {
0048 |       name,
0049 |       embeddingModel,
0050 |       rerankModel,
0051 |       visionModel,
0052 |       documentParser,
0053 |     }: {
0054 |       name: string
0055 |       embeddingModel: string
0056 |       rerankModel: string
0057 |       visionModel?: string
0058 |       documentParser?: { type: string; mineru?: { apiToken: string } }
0059 |     }
0060 |   ) => {
0061 |     try {
0062 |       log.info(
0063 |         `ipcMain: kb:create, name=${name}, embeddingModel=${embeddingModel}, r
0064 |       )
0065 |
0066 |       // Validate required fields
0067 |       if (!name || !name.trim()) {
0068 |         throw new Error('Knowledge base name is required')
0069 |       }
0070 |       if (!embeddingModel || !embeddingModel.trim()) {
0071 |         throw new Error('Embedding model is required')
0072 |       }
0073 |
0074 |       const db = getDatabase()
0075 |       const documentParserJson = documentParser ? JSON.stringify(documentParse

```

```

0076 |         const rs = await db.execute({
0077 |             sql: 'INSERT INTO knowledge_base (name, embedding_model, rerank_model,
0078 |             args: [name.trim(), embeddingModel, rerankModel || null, visionModel |
0079 |         })
0080 |         const id = rs.lastInsertRowid
0081 |
0082 |         if (!id) {
0083 |             throw new Error('Failed to create knowledge base')
0084 |         }
0085 |
0086 |         log.info(`[IPC] Knowledge base created successfully: id=${id}, name=${na
0087 |         return { id, name: name.trim() }
0088 |     } catch (error: any) {
0089 |         log.error(`ipcMain: kb:create failed for name=${name}`, error)
0090 |         sentry.withScope((scope) => {
0091 |             scope.setTag('component', 'knowledge-base-ipc')
0092 |             scope.setTag('operation', 'kb_create')
0093 |             scope.setExtra('name', name)
0094 |             scope.setExtra('embeddingModel', embeddingModel)
0095 |             scope.setExtra('rerankModel', rerankModel)
0096 |             scope.setExtra('visionModel', visionModel)
0097 |             scope.setExtra('documentParser', documentParser?.type)
0098 |             sentry.captureException(error)
0099 |         })
0100 |         throw error
0101 |     }
0102 | }
0103 | )
0104 |
0105 | ipcMain.handle(
0106 |     'kb:update',
0107 |     async (
0108 |         _event,
0109 |         { id, name, rerankModel, visionModel }: { id: number; name?: string; reran
0110 |     ) => {
0111 |         try {
0112 |             log.info(`ipcMain: kb:update, id=${id}, name=${name}, rerankModel=${rera
0113 |
0114 |             if (!id || id <= 0) {
0115 |                 throw new Error('Invalid knowledge base ID')
0116 |             }
0117 |
0118 |             if (!name && rerankModel === undefined && visionModel === undefined) {
0119 |                 return 0
0120 |             }
0121 |
0122 |             const db = getDatabase()
0123 |             let sql = 'UPDATE knowledge_base SET '
0124 |             const args: (string | number)[] = []
0125 |

```



```

0126 |         if (name !== undefined) {
0127 |             if (!name.trim()) {
0128 |                 throw new Error('Knowledge base name cannot be empty')
0129 |             }
0130 |             sql += 'name = ?'
0131 |             args.push(name.trim())
0132 |         }
0133 |         if (rerankModel !== undefined) {
0134 |             if (args.length > 0) sql += ', '
0135 |             sql += 'rerank_model = ?'
0136 |             args.push(rerankModel ?? '')
0137 |         }
0138 |         if (visionModel !== undefined) {
0139 |             if (args.length > 0) sql += ', '
0140 |             sql += 'vision_model = ?'
0141 |             args.push(visionModel ?? '')
0142 |         }
0143 |         sql += ' WHERE id = ?'
0144 |         args.push(id)
0145 |
0146 |         // cannot do commit db operation here, because sql is not complete
0147 |         log.info(`[IPC] Knowledge base updated: id=${id}, affected rows=${rs.row
0148 |         return rs.rowsAffected
0149 |     } catch (error: any) {
0150 |         log.error(`ipcMain: kb:update failed for id=${id}`, error)
0151 |         sentry.withScope((scope) => {
0152 |             scope.setTag('component', 'knowledge-base-ipc')
0153 |             scope.setTag('operation', 'kb_update')
0154 |             scope.setExtra('kbId', id)
0155 |             scope.setExtra('name', name)
0156 |             scope.setExtra('rerankModel', rerankModel)
0157 |             scope.setExtra('visionModel', visionModel)
0158 |             sentry.captureException(error)
0159 |         })
0160 |         throw error
0161 |     }
0162 | }
0163 | )
0164 |
0165 | ipcMain.handle('kb:delete', async (_event, kbId: number): Promise<{ success
0166 |     try {
0167 |         log.info(`ipcMain: kb:delete, kbId=${kbId}`)
0168 |
0169 |         if (!kbId || kbId <= 0) {
0170 |             throw new Error('Invalid knowledge base ID')
0171 |         }
0172 |
0173 |         await withTransaction(async () => {
0174 |             const db = getDatabase()
0175 |             const vectorStore = getVectorStore()

```

```

0121 |
0122 | // Message content part schemas
0123 | export const MessageTextPartSchema = z.object({
0124 |   type: z.literal('text'),
0125 |   text: z.string(),
0126 | })
0127 |
0128 | export const MessageImagePartSchema = z.object({
0129 |   type: z.literal('image'),
0130 |   url: z.string(),
0131 | })
0132 |
0133 |
0134 | export const MessageInfoPartSchema = z.object({
0135 |   type: z.literal('info'),
0136 |   text: z.string(),
0137 |   values: z.record(z.string(), z.unknown()).optional(),
0138 | })
0139 |
0140 | export const MessageReasoningPartSchema = z.object({
0141 |   type: z.literal('reasoning'),
0142 |   text: z.string(),
0143 |   startTime: z.number().optional(),
0144 |   duration: z.number().optional(),
0145 | })
0146 |
0147 | export const MessageToolCallPartSchema = z.object({
0148 |   type: z.literal('tool-call'),
0149 |   state: z.enum(['call', 'result', 'error']),
0150 |   toolCallId: z.string(),
0151 |   toolName: z.string(),
0152 |   arguments: z.unknown().optional(),
0153 | })
0154 | })
0155 |
0156 | export const MessageContentPartSchema = z.discriminatedUnion(
0157 |   'type', [
0158 |     MessageTextPartSchema,
0159 |     MessageImagePartSchema,
0160 |     MessageInfoPartSchema,
0161 |     MessageReasoningPartSchema,
0162 |     MessageToolCallPartSchema,
0163 |   ])
0164 | export const MessageContentPartsSchema = z.array(MessageContentPartSchema)
0165 |
0166 | export const StreamTextResultSchema = z.object({
0167 |   contentParts: MessageContentPartsSchema,
0168 |   reasoningContent: z.string().optional(),
0169 |   usage: z.custom<LanguageModelUsage>().optional(),
0170 |   finishReason: z.string().optional(),

```

```

0171 | })
0172 |
0173 | // Tool and provider schemas
0174 | export const ToolUseScopeSchema = z.enum(['web-browsing', 'knowledge-base', 'rea
0175 |
0176 | export const ModelProviderSchema = z.union([z.nativeEnum(ModelProviderEnum), z.s
0177 |
0178 | // Message status schemas
0179 | export const MessageStatusSchema = z.discriminatedUnion('type', [
0180 |   z.object({
0181 |     type: z.literal('sending_file'),
0182 |     mode: z.enum(['local', 'advanced']).optional(),
0183 |   }),
0184 |   z.object({
0185 |     type: z.literal('loading_webpage'),
0186 |     mode: z.enum(['local', 'advanced']).optional(),
0187 |   }),
0188 |   z.object({
0189 |     type: z.literal('retrying'),
0190 |     attempt: z.number(),
0191 |     maxAttempts: z.number(),
0192 |     error: z.string().optional(),
0193 |   }),
0194 | ])
0195 |
0196 | // Main Message schema
0197 | // Define a custom function type for cancel
0198 | const CancelFunctionSchema = z.custom<(() => void) | undefined>({
0199 |   (val) => val === undefined || typeof val === 'function',
0200 |   { message: 'Must be a function or undefined' }
0201 | })
0202 |
0203 | const MessageUsageSchema = z.object({
0204 |   inputTokens: z.number().optional().catch(undefined),
0205 |   /**
0206 |    The number of output (completion) tokens used.
0207 |    */
0208 |   outputTokens: z.number().optional().catch(undefined),
0209 |   /**
0210 |    The total number of tokens as reported by the provider.
0211 |    This number might be different from the sum of `inputTokens` and `outputTokens`
0212 |    and e.g. include reasoning tokens or other overhead.
0213 |    */
0214 |   totalTokens: z.number().optional().catch(undefined),
0215 |   /**
0216 |    The number of reasoning tokens used.
0217 |    */
0218 |   reasoningTokens: z.number().optional().catch(undefined),
0219 |   /**
0220 |    The number of cached input tokens.

```

```

0221 |     */
0222 |     cachedInputTokens: z.number().optional().catch(undefined),
0223 | })
0224 |
0225 | export const MessageSchema = z.object({
0226 |   id: z.string(),
0227 |   role: z.nativeEnum(MessageRoleEnum),
0228 |   name: z.string().optional(),
0229 |   cancel: CancelFunctionSchema.optional(),
0230 |   generating: z.boolean().optional(),
0231 |   aiProvider: z.union([ModelProviderSchema, z.string()]).optional(),
0232 |   model: z.string().optional(),
0233 |   style: z.string().optional(),
0234 |   files: z.array(MessageFileSchema).optional(),
0235 |   images: z.array(MessageImageSchema).optional(),
0236 |   reasoningContent: z.string().optional().describe('deprecated, moved to content
0237 |   contentParts: MessageContentPartsSchema,
0238 |   isStreamingMode: z.boolean().optional(),
0239 |   errorCode: z.number().optional(),
0240 |   error: z.string().optional(),
0241 |   errorExtra: z.record(z.string(), z.unknown()).optional(),
0242 |   status: z.array(MessageStatusSchema).optional(),
0243 |   wordCount: z.number().optional(),
0244 |   tokenCount: z.number().optional(), // output token count
0245 |   tokensUsed: z.number().optional(), // deprecated, use `usage` instead
0246 |   usage: MessageUsageSchema.optional().catch(undefined),
0247 |   timestamp: z.number().optional(),
0248 |   firstTokenLatency: z.number().optional(),
0249 |   finishReason: z.string().optional(),
0250 |   tokenCountMap: TokenCountMapSchema.optional(), // estimate token count as input
0251 |   tokenCalculatedAt: TokenCalculatedAtSchema,
0252 |   updatedAt: z.number().optional(),
0253 |   isSummary: z.boolean().optional(), // Marks message as a compaction summary
0254 | })
0255 |
0256 | // Compaction point schema (for context management)
0257 | export const CompactionPointSchema = z.object({
0258 |   summaryMessageId: z.string(),
0259 |   boundaryMessageId: z.string(),
0260 |   createdAt: z.number(),
0261 | })
0262 |
0263 | // Session schemas
0264 | export const SessionTypeSchema = z.enum(['chat', 'picture'])
0265 |
0266 | export const MessageForkListSchema = z.object({
0267 |   id: z.string(),
0268 |   messages: z.array(MessageSchema),
0269 | })
0270 |

```

```

0271 | export const MessageForkSchema = z.object({
0272 |   position: z.number(),
0273 |   lists: z.array(MessageForkListSchema),
0274 |   createdAt: z.number(),
0275 | })
0276 |
0277 | export const SessionThreadSchema = z.object({
0278 |   id: z.string(),
0279 |   name: z.string(),
0280 |   messages: z.array(MessageSchema),
0281 |   createdAt: z.number(),
0282 |   compactionPoints: z.array(CompactionPointSchema).optional(),
0283 | })
0284 |
0285 | export const SessionSchema = z.object({
0286 |   id: z.string(),
0287 |   type: SessionTypeSchema.optional(),
0288 |   name: z.string(),
0289 |   picUrl: z.string().optional(),
0290 |   messages: z.array(MessageSchema),
0291 |   starred: z.boolean().optional(),

```

st (e.g., migrated pi

.optional(),
(),

```

0291 |   starred: z.boolean().optional(),
0292 |   hidden: z.boolean().optional(), // Hidden from session li
0293 |   copilotId: z.string().optional(),
0294 |   assistantAvatarKey: z.string().optional(),
0295 |   settings: SessionSettingsSchema.optional(),
0296 |   threads: z.array(SessionThreadSchema).optional(),
0297 |   threadName: z.string().optional(),
0298 |   messageForksHash: z.record(z.string(), MessageForkSchema)
0299 |   compactionPoints: z.array(CompactionPointSchema).optional
0300 | })
0301 |
0302 | export const SessionMetaSchema = SessionSchema.pick({
0303 |   id: true,
0304 |   name: true,
0305 |   starred: true,
0306 |   hidden: true,
0307 |   assistantAvatarKey: true,
0308 |   picUrl: true,
0309 |   type: true,
0310 | })
0311 |
0312 | export const SessionThreadBriefSchema = z.object({
0313 |   id: z.string(),
0314 |   name: z.string(),
0315 |   createdAt: z.number().optional(),
0316 |   createdAtLabel: z.string().optional(),
0317 |   firstMessageId: z.string(),
0318 |   messageCount: z.number(),
0319 | })
0320 |

```



```

0321 | // Export types inferred from schemas
0322 | export type SearchResultItem = z.infer<typeof SearchResultItemSchema>
0323 | export type SearchResult = z.infer<typeof SearchResultSchema>
0324 | export type MessageFile = z.infer<typeof MessageFileSchema>
0325 | export type MessageLink = z.infer<typeof MessageLinkSchema>
0326 | export type MessagePicture = z.infer<typeof MessagePictureSchema>
0327 | export type MessageTextPart = z.infer<typeof MessageTextPartSchema>
0328 | export type MessageImagePart = z.infer<typeof MessageImagePartSchema>
0329 | export type MessageInfoPart = z.infer<typeof MessageInfoPartSchema>
0330 | export type MessageReasoningPart = z.infer<typeof MessageReasoningPartSchema>
0331 | export type MessageToolCallPart<Args = unknown, Result = unknown> = z.infer<type
0332 |   args: Args
0333 |   result?: Result
0334 | }
0335 | export type MessageContentPart = z.infer<typeof MessageContentPartSchema>
0336 | export type StreamMessageResult = z.infer<typeof StreamMessageResultSchema>
0337 | export type UserMessage = z.infer<typeof UserMessageSchema>
0338 | export type ModelProvider = z.infer<typeof ModelProviderSchema>
0339 | export type ConversationInfo = z.infer<typeof ConversationInfoSchema>
0340 | export type Message = z.infer<typeof MessageSchema>
0341 | export type SessionType = z.infer<typeof SessionTypeSchema>
0342 | export type ConversationPoint = z.infer<typeof ConversationPointSchema>
0343 | export type Session = z.infer<typeof SessionSchema>
0344 | export type SessionMeta = z.infer<typeof SessionMetaSchema>
0345 | export type SessionThread = z.infer<typeof SessionThreadSchema>
0346 | export type SessionThreadBrief = z.infer<typeof SessionThreadBriefSchema>
// ===== END: Unexported Types =====
// ===== FUTURE: unexported types =====
0361 | import { * } from 'nod'
0362 | import { ModelProviderEnum, ModelProviderType } from './provider'
0363 |
0364 | // Re-export for backward compatibility
0365 | export { ModelProviderType } from './provider'
0366 |
0367 | // ===== Document Parser Types =====
0368 | /**
0369 |  * Document parser service type
0370 |  * = none: No parsing service, only exports basic text files (mobile/web default)
0371 |  * = none: No parsing service, only exports basic text files (mobile/web default)
0372 |  * = none: No parsing service, only exports basic text files (mobile/web default)
0373 |  * = minor: Third-party WinRT parsing service (desktop only)
0374 |  */
0375 | export type DocumentParserType = 'none' | 'local' | 'winrt'
0376 |
0377 | export const DocumentParserSchema = z.object({
0378 |   type: z.enum(['none', 'local', 'winrt']),
0379 |   minor: z.optional(z.boolean()),
0380 |   object: z.optional(z.boolean()),
0381 |   application: z.optional(z.boolean()),
0382 | })

```

```

0023 |     .optional(),
0024 | })
0025 |
0026 | export type DocumentParserConfig = z.infer<typeof DocumentParserConfigSchema>
0027 |
0028 | export const DEFAULT_DOCUMENT_PARSER_CONFIG: DocumentParserConfig = {
0029 |   type: 'local',
0030 | }
0031 |
0032 | export const ProviderModelInfoSchema = z.object({
0033 |   modelId: z.string(),
0034 |   type: z.enum(['chat', 'embedding', 'rerank']).optional().catch(undefined),
0035 |   apiStyle: z.enum(['google', 'openai', 'anthropic']).optional().catch(undefined),
0036 |   nickname: z.string().optional().catch(undefined),
0037 |   labels: z.array(z.string()).optional().catch([]),
0038 |   capabilities: z
0039 |     .array(z.enum(['vision', 'reasoning', 'tool_use', 'web_search']))
0040 |     .optional()
0041 |     .catch([]),
0042 |   contextWindow: z.number().optional().catch(undefined),
0043 |   maxOutput: z.number().optional().catch(undefined),
0044 | })
0045 |
0046 | export const ProviderSettingsSchema = z.object({
0047 |   apiKey: z.string().optional().catch(undefined),
0048 |   apiHost: z.string().optional().catch(undefined),
0049 |   apiPath: z.string().optional().catch(undefined),
0050 |   models: z.array(ProviderModelInfoSchema).optional().catch(undefined),
0051 |   excludedModels: z.array(z.string()).optional().catch(undefined),
0052 |   useProxy: z.boolean().optional().catch(undefined),
0053 |   wanjieWorkerHost: z.string().optional().catch(undefined),
0054 |   wanjieEncryptionKey: z.string().optional().catch(undefined),
0055 |   wanjiePhone: z.string().optional().catch(undefined),
0056 |   wanjieSmsId: z.string().optional().catch(undefined),
0057 |   wanjieSmsCooldownUntil: z.number().optional().catch(undefined),
0058 |   wanjieAccountToken: z.string().optional().catch(undefined),
0059 |
0060 |   // azure
0061 |   endpoint: z.string().optional().catch(undefined),
0062 |   deploymentName: z.string().optional().catch(undefined),
0063 |   dalleDeploymentName: z.string().optional().catch(undefined),
0064 |   apiVersion: z.string().optional().catch(undefined),
0065 | })
0066 |
0067 | const BuiltinProviderBaseInfoSchema = z.object({
0068 |   id: z.nativeEnum(ModelProviderEnum),
0069 |   name: z.string(),
0070 |   type: z.nativeEnum(ModelProviderType).catch(ModelProviderType.OpenAI),
0071 |   isCustom: z.literal(false).optional().catch(undefined),
0072 |   description: z.string().optional().catch(undefined),

```

```

0073 |   urls: z
0074 |     .object({
0075 |       website: z.string().nullish(),
0076 |       apiKey: z.string().nullish(),
0077 |       docs: z.string().nullish(),
0078 |       models: z.string().nullish(),
0079 |     })
0080 |     .optional()
0081 |     .catch(undefined),
0082 |   defaultSettings: ProviderSettingsSchema.optional().catch(undefined),
0083 | })
0084 |
0085 | const CustomProviderBaseInfoSchema = BuiltinProviderBaseInfoSchema.extend({
0086 |   id: z.string(),
0087 |   iconUrl: z.string().optional().catch(undefined),
0088 |   isCustom: z.literal(true),
0089 | })
0090 |
0091 | const ProviderBaseInfoSchema = z.discriminatedUnion('isCustom', [
0092 |   BuiltinProviderBaseInfoSchema,
0093 |   CustomProviderBaseInfoSchema,
0094 | ])
0095 |
0096 | const ClaudeParamsSchema = z.object({
0097 |   thinking: z.object({
0098 |     type: z.enum(['enabled', 'disabled']).default('enabled'),
0099 |     budgetTokens: z.number().catch(1024),
0100 |   }),
0101 | })
0102 |
0103 | const OpenAIParamsSchema = z.object({
0104 |   reasoningEffort: z.enum(['low', 'medium', 'high']).optional().catch(undefined)
0105 | })
0106 |
0107 | const GoogleParamsSchema = z.object({
0108 |   thinkingConfig: z.object({
0109 |     thinkingBudget: z.number().catch(1024),
0110 |     includeThoughts: z.boolean().catch(true),
0111 |   }),
0112 | })
0113 |
0114 | export const ProviderOptionsSchema = z.object({
0115 |   claude: ClaudeParamsSchema.optional(),
0116 |   openai: OpenAIParamsSchema.optional(),
0117 |   google: GoogleParamsSchema.optional(),
0118 | })
0119 |
0120 | // NOTICE: Global settings is for new session default settings, set to session w
0121 | export const GlobalSessionSettingsSchema = z.object({
0122 |   maxContextMessageCount: z.number().optional().catch(undefined),

```

```

0123 |   temperature: z.number().optional().catch(undefined),
0124 |   topP: z.number().optional().catch(undefined),
0125 |   maxTokens: z.number().optional().catch(undefined),
0126 |   stream: z.boolean().optional().catch(true),
0127 | })
0128 |
0129 | export const SessionSettingsSchema = GlobalSessionSettingsSchema.extend({
0130 |   provider: z.string().optional().catch(undefined),
0131 |   modelId: z.string().optional().catch(undefined),
0132 |   dalleStyle: z.enum(['vivid', 'natural']).optional().catch('vivid'),
0133 |   imageGenerateNum: z.number().optional().catch(1),
0134 |   providerOptions: ProviderOptionsSchema.optional().catch(undefined),
0135 |   autoCompaction: z.boolean().optional().catch(undefined),
0136 | })
0137 |
0138 | const UnifiedTokenUsageDetailSchema = z.object({
0139 |   type: z.string(), // "plan" | "trial" | ... (more types in future)
0140 |   token_usage: z.number(),
0141 |   token_limit: z.number(),
0142 | })
0143 |
0144 | const RemoteLicensesDetailSchema = z.object({
0145 |   type: z.string().optional(),
0146 |   name: z.string(),
0147 |   status: z.string().optional(),
0148 |   defaultModel: z.string().optional(),
0149 |   remaining_quotes_3s: z.number(),
0150 |   remaining_quotes_4s: z.number(),
0151 |   remaining_quotes_images: z.number(),
0152 |   image_used_count: z.number(),
0153 |   image_total_quotes: z.number(),
0154 |   plan_image_limit: z.number(),
0155 |   token_refresh_time: z.string(),
0156 |   token_next_refresh_time: z.string().optional(),
0157 |   token_expire_time: z.string().nullish(),
0158 |   remaining_quotes_unified: z.number(),
0159 |   expansion_pack_limit: z.number(),
0160 |   expansion_pack_usage: z.number(),
0161 |   unified_token_usage: z.number(),
0162 |   unified_token_limit: z.number(),
0163 |   unified_token_usage_details: z.array(UnifiedTokenUsageDetailSchema).default([]),
0164 |   key: z.string().optional(),
0165 |   price_type: z.string().optional(),
0166 |   order_type: z.string().optional(),
0167 |   utm_source: z.string().optional(),
0168 |   expires_at: z.string().optional(),
0169 |   recurring_payment: z.boolean().nullish(),
0170 |   payment_type: z.string().optional(),
0171 | })
0172 |

```



```

0173 | export const shortcutSendValues = [
0174 |   '',
0175 |   'Enter',
0176 |   'Ctrl+Enter',
0177 |   'Command+Enter',
0178 |   'Shift+Enter',
0179 |   'Ctrl+Shift+Enter',
0180 |   'CommandOrControl+Enter',
0181 | ]
0182 | const ShortcutSendValueSchema = z.enum(shortcutSendValues as [string, ...string[
0183 |
0184 | export const shortcutToggleWindowValues = ['', 'Alt+`', 'Alt+Space', 'Ctrl+Alt+S
0185 | const ShortcutToggleWindowValueSchema = z.enum(shortcutToggleWindowValues as [st
0186 |
0187 | const ShortcutSettingSchema = z.object({
0188 |   quickToggle: ShortcutToggleWindowValueSchema,
0189 |   inputBoxFocus: z.string(),
0190 |   inputBoxWebBrowsingMode: z.string(),
0191 |   newChat: z.string(),
0192 |   newPictureChat: z.string(),
0193 |   sessionListNavNext: z.string(),
0194 |   sessionListNavPrev: z.string(),
0195 |   sessionListNavTargetIndex: z.string(),
0196 |   messageListRefreshContext: z.string(),
0197 |   dialogOpenSearch: z.string(),
0198 |   optionNavUp: z.string(),
0199 |   optionNavDown: z.string(),
0200 |   optionSelect: z.string(),
0201 |   inputBoxSendMessage: ShortcutSendValueSchema,
0202 |   inputBoxSendMessageWithoutResponse: ShortcutSendValueSchema,
0203 | })
0204 |
0205 | const ExtensionSettingsSchema = z.object({
0206 |   webSearch: z.object({
0207 |     provider: z.enum(['build-in', 'bing', 'baidu', 'tavily']),
0208 |     tavilyApiKey: z.string().optional(),
0209 |     tavilySearchDepth: z.string().optional(),
0210 |     tavilyMaxResults: z.number().optional(),
0211 |     tavilyTimeRange: z.string().optional(),
0212 |     tavilyIncludeRawContent: z.string().optional(),
0213 |   }),
0214 |   knowledgeBase: z
0215 |     .object({
0216 |       models: z.object({
0217 |         embedding: z
0218 |           .object({
0219 |             modelId: z.string(),
0220 |             providerId: z.string(),
0221 |           })
0222 |         .nullable()

```



```

0223 |         .optional(),
0224 |         rerank: z
0225 |         .object({
0226 |             modelId: z.string(),
0227 |             providerId: z.string(),
0228 |         })
0229 |         .nullable()
0230 |         .optional(),
0231 |     })),
0232 | })
0233 |     .optional(),
0234 |     // Document parser configuration for global default
0235 |     documentParser: DocumentParserConfigSchema.optional(),
0236 | })
0237 |
0238 | const MCPTransportConfigSchema = z.discriminatedUnion('type', [
0239 |     z.object({
0240 |         type: z.literal('stdio'),
0241 |         command: z.string(),
0242 |         args: z.array(z.string()),
0243 |         env: z.record(z.string(), z.string()).optional(),
0244 |     }),
0245 |     z.object({
0246 |         type: z.literal('http'),
0247 |         url: z.string(),
0248 |         headers: z.record(z.string(), z.string()).optional(),
0249 |     }),
0250 | ])
0251 |
0252 | const MCPServerConfigSchema = z.object({
0253 |     id: z.string(),
0254 |     name: z.string(),
0255 |     enabled: z.boolean(),
0256 |     transport: MCPTransportConfigSchema,
0257 | })
0258 |
0259 | const MCPSettingsSchema = z.object({
0260 |     servers: z.array(MCPServerConfigSchema),
0261 |     enabledBuiltinServers: z.array(z.string()),
0262 | })
0263 |
0264 | export enum Theme {
0265 |     Dark,
0266 |     Light,
0267 |     System,
0268 | }
0269 |
0270 | export const SettingsSchema = GlobalSessionSettingsSchema.extend({
0271 |     providers: z.record(z.string(), ProviderSettingsSchema).optional().catch(undefi
0272 |     customProviders: z.array(CustomProviderBaseInfoSchema).optional().catch(undefi

```

```

0273 |   favoritedModels: z
0274 |     .array(
0275 |       z.object({
0276 |         provider: z.string(),
0277 |         model: z.string(),
0278 |       })
0279 |     )
0280 |     .optional()
0281 |     .catch(undefined),
0282 |
0283 |   // default models
0284 |   defaultChatModel: z
0285 |     .object({
0286 |       provider: z.string(),
0287 |       model: z.string(),
0288 |     })
0289 |     .optional()
0290 |     .catch(undefined),
0291 |   threadNamingModel: z
0292 |     .object({
0293 |       provider: z.string(),
0294 |       model: z.string(),
0295 |     })
0296 |     .optional()
0297 |     .catch(undefined),
0298 |   searchTermConstructionModel: z
0299 |     .object({
0300 |       provider: z.string(),
0301 |       model: z.string(),
0302 |     })
0303 |     .optional()
0304 |     .catch(undefined),
0305 |   ocrModel: z
0306 |     .object({
0307 |       provider: z.string(),
0308 |       model: z.string(),
0309 |     })
0310 |     .optional()
0311 |     .catch(undefined),
0312 |
0313 |   // license settings
0314 |   licenseKey: z.string().optional(),
0315 |   licenseInstances: z.record(z.string(), z.string()).optional().catch(undefined),
0316 |   licenseDetail: RemoteLicenseDetailSchema.optional().catch(undefined),
0317 |   licenseActivationMethod: z.enum(['login', 'manual']).optional(),
0318 |   lastSelectedLicenseByUser: z.record(z.string(), z.string()).optional().catch(u
0319 |   // In Jira, z.string() in UI shows 'login' as the default, but in the API, it's 'manual'.
0320 |   memorizedManualLicenseKey: z.string().optional(),
0321 |
0322 |   // chat settings

```

```

0323 | showWordCount: z.boolean().optional().catch(undefined),
0324 | showTokenCount: z.boolean().optional().catch(undefined),
0325 | showTokenUsed: z.boolean().optional().catch(undefined),
0326 | showModelName: z.boolean().optional().catch(undefined),
0327 | showMessageTimestamp: z.boolean().optional().catch(undefined),
0328 | showFirstTokenLatency: z.boolean().optional().catch(undefined),
0329 |
0330 | theme: z.nativeEnum(Theme),
0331 | language: z.enum([
0332 |   'en',
0333 |   'zh-Hans',
0334 |   'zh-Hant',
0335 |   'ja',
0336 |   'ko',
0337 |   'ru',
0338 |   'de',
0339 |   'fr',
0340 |   'pt-PT',
0341 |   'es',
0342 |   'ar',
0343 |   'it-IT',
0344 |   'sv',
0345 |   'nb-NO',
0346 | ]),
0347 | languageInitd: z.boolean().optional(),
0348 | fontSize: z.number().catch(14),
0349 | spellCheck: z.boolean().optional(),
0350 |
0351 | startupPage: z.enum(['home', 'session']).optional(),
0352 |

```

0353 | ~~enableMarkdownRendering: z.boolean(), // 是否关闭快捷切换窗口显示 (李思 为了兼容历史数据 这个字~~

段永远不要使

```

0354 |
0355 | defaultPrompt: z.string().optional(), // 新会话的默认 prompt
0356 |
0357 | proxy: z.string().optional(), // 代理地址
0358 |
0359 | allowReportingAndTracking: z.boolean().optional(), // 是否允许错误报告和事件追踪
0360 |
0361 | userAvatarKey: z.string().optional(), // 用户头像的 key
0362 | defaultAssistantAvatarKey: z.string().optional(), // 默认助手头像的 key
0363 |
0364 | enableMarkdownRendering: z.boolean().default(true),
0365 | enableMermaidRendering: z.boolean().default(true),
0366 | enableLaTeXRendering: z.boolean().default(true),
0367 | injectDefaultMetadata: z.boolean().default(true), // 是否注入默认附加元数据 (如模型名称、当前日期)
0368 | autoPreviewArtifacts: z.boolean().default(false), // 是否自动展开预览 artifacts
0369 | autoCollapseCodeBlock: z.boolean().default(true), // 是否自动折叠代码块
0370 | pasteLongTextAsAFile: z.boolean().default(true), // 是否将长文本粘贴为文件
0371 |
0372 | autoGenerateTitle: z.boolean().default(true),

```

```

0373 |
0374 |   autoCompaction: z.boolean().default(true),
0375 |   compactionThreshold: z.number().min(0.4).max(0.9).default(0.6),
0376 |
0377 |   autoLaunch: z.boolean().default(false),
0378 |   autoUpdate: z.boolean().default(true), // 是否自动检查更新
0379 |   betaUpdate: z.boolean().default(false), // 是否自动检查 beta 更新
0380 |
0381 |   shortcutSettings: ShortcutSettingsSchema,
0382 |
0383 |   extension: ExtensionSettingsSchema,
0384 |   mcp: MCPSettingsSchema,
0385 | })
0386 |
0387 | // TODO: provider 的 base info 和 settings 混在一起了, 可以考虑像 session settings 和 global se
0388 | export type ProviderInfo = (ProviderBaseInfo | CustomProviderBaseInfo) & Provide
0389 |
0390 | export type SessionSettings = z.infer<typeof SessionSettingsSchema>
0391 | export type Settings = z.infer<typeof SettingsSchema>
0392 | export type ProviderModelInfo = z.infer<typeof ProviderModelInfoSchema>
0393 | export type ProviderBaseInfo = z.infer<typeof ProviderBaseInfoSchema>
0394 | export type ProviderSettings = z.infer<typeof ProviderSettingsSchema>
0395 | export type BuiltinProviderBaseInfo = z.infer<typeof BuiltinProviderBaseInfoSche
0396 | export type CustomProviderBaseInfo = z.infer<typeof CustomProviderBaseInfoSchema
0397 | export type ClaudeParams = z.infer<typeof ClaudeParamsSchema>
0398 | export type OpenAIParams = z.infer<typeof OpenAIParamsSchema>
0399 | export type GoogleParams = z.infer<typeof GoogleParamsSchema>
0400 | export type ProviderOptions = z.infer<typeof ProviderOptionsSchema>
0401 | export type GlobalSessionSettings = z.infer<typeof GlobalSessionSettingsSchema>
0402 | export type RemoteLicenseDetail = z.infer<typeof RemoteLicenseDetailSchema>
0403 | export type UnifiedTokenUsageDetail = z.infer<typeof UnifiedTokenUsageDetailsche
0404 | export type ShortcutSendValue = z.infer<typeof ShortcutSendValueSchema>
0405 | export type ShortcutToggleWindowValue = z.infer<typeof ShortcutToggleWindowValue
0406 | export type ShortcutName = keyof ShortcutSetting
0407 | export type ShortcutSetting = z.infer<typeof ShortcutSettingSchema>
0408 | export type ExtensionSettings = z.infer<typeof ExtensionSettingsSchema>
0409 | export type MCPTransportConfig = z.infer<typeof MCPTransportConfigSchema>
0410 | export type MCPServerConfig = z.infer<typeof MCPServerConfigSchema>
0411 | export type MCPSettings = z.infer<typeof MCPSettingsSchema>
// ===== END FILE: src/shared/types/settings.ts =====
// ===== FILE: src/shared/types.ts =====
0001 | import { v4 as uuidv4 } from 'uuid'
0002 | import {
0003 |   type CompactionPoint,
0004 |   type Message,
0005 |   type MessageRole,
0006 |   MessageRoleEnum,
0007 |   type Session,
0008 |   type SessionThread,
0009 |   type TokenCountMap,

```

```

0010 | } from './types/session'
0011 | import type { DocumentParserConfig, DocumentParserType } from './types/settings'
0012 |
0013 | export type Updater<T extends object> = Partial<T> | UpdaterFn<T>
0014 | export type UpdaterFn<T extends object> = (data: T | null | undefined) => T
0015 |
0016 | export type MessageTokenCountResult = { id: string; tokenCountMap: TokenCountMap
0017 |
0018 | export type SettingWindowTab = 'all' | 'display' | 'chat' | 'advanced' | 'extensi
0019 |
0020 | export type ExportChatScope = 'all_threads' | 'current_thread'
0021 |
0022 | export type ExportChatFormat = 'Markdown' | 'TXT' | 'HTML'
0023 |
0024 | export function isChatSession(session: Session) {
0025 |   return session.type === 'chat' || !session.type
0026 | }
0027 | export function isPictureSession(session: Session) {
0028 |   return session.type === 'picture'
0029 | }
0030 |
0031 | export function createMessage(role: MessageRole = MessageRoleEnum.User, content:
0032 |   return {
0033 |     id: uuidv4(),
0034 |     contentParts: content ? [{ type: 'text', text: content }] : [],
0035 |     role: role,
0036 |     timestamp: Date.now(),
0037 |   }
0038 | }
0039 |
0040 | export type Language =
0041 |   | 'en'
0042 |   | 'zh-Hans'
0043 |   | 'zh-Hant'
0044 |   | 'ja'
0045 |   | 'ko'
0046 |   | 'ru'
0047 |   | 'de'
0048 |   | 'fr'
0049 |   | 'pt-PT'
0050 |   | 'es'
0051 |   | 'ar'
0052 |   | 'it-IT'
0053 |   | 'sv'
0054 |   | 'nb-NO'
0055 |
0056 | export interface Config {
0057 |   uuid: string
0058 | }
0059 |

```



```
0060 | export interface SponsorAd {
0061 |   text: string
0062 |   url: string
0063 | }
0064 |
0065 | export interface SponsorAboutBanner {
0066 |   type: 'picture' | 'picture-text'
0067 |   name: string
0068 |   pictureUrl: string
0069 |   link: string
0070 |   title: string
0071 |   description: string
0072 | }
0073 |
0074 | export interface CopilotDetail {
0075 |   id: string
0076 |   name: string
0077 |   picUrl?: string
0078 |   prompt: string
0079 |   demoQuestion?: string
0080 |   demoAnswer?: string
0081 |   starred?: boolean
0082 |   usedCount: number
0083 |   shared?: boolean
0084 | }
0085 |
0086 | export interface Toast {
0087 |   id: string
0088 |   content: string
0089 |   duration?: number
0090 | }
0091 |
0092 | export interface RemoteConfig {
0093 |   current_version: string
0094 |   product_ids: number[]
0095 | }
0096 |
0097 | export function copyMessage(source: Message): Message {
0098 |   return {
0099 |     ...source,
0100 |     cancel: undefined,
0101 |     id: uuidv4(),
0102 |   }
0103 | }
0104 |
0105 | export function copyMessagesWithMapping(messages: Message[]): {
0106 |   messages: Message[]
0107 |   idMapping: Map<string, string>
0108 | } {
0109 |   const idMapping = new Map<string, string>()
```

```

0110 |   const newMessages = messages.map((msg) => {
0111 |     const newMsg = copyMessage(msg)
0112 |     idMapping.set(msg.id, newMsg.id)
0113 |     return newMsg
0114 |   })
0115 |   return { messages: newMessages, idMapping }
0116 | }
0117 |
0118 | export function copyThreads(source?: SessionThread[], idMapping?: Map<string, st
0119 |   if (!source) {
0120 |     return undefined
0121 |   }
0122 |   return source.map((thread) => {
0123 |     // Use copyMessagesWithMapping for thread messages
0124 |     const { messages: newMessages, idMapping: threadIdMapping } = copyMessagesWi
0125 |
0126 |     // Combine external mapping (if provided) with thread mapping
0127 |     const combinedMapping = idMapping ? new Map([...idMapping, ...threadIdMappin
0128 |
0129 |     // Map compactionPoints (if they exist)
0130 |     const newCompactionPoints = thread.compactionPoints
0131 |       ? .map((cp) => {
0132 |         const newSummaryId = combinedMapping.get(cp.summaryMessageId)
0133 |         const newBoundaryId = combinedMapping.get(cp.boundaryMessageId)
0134 |         // Skip compactionPoints with unmapped IDs
0135 |         if (!newSummaryId || !newBoundaryId) {
0136 |           console.warn('[copyThreads] Skipping compactionPoint with unmapped IDs
0137 |           return null
0138 |         }
0139 |         return {
0140 |           ...cp,
0141 |           summaryMessageId: newSummaryId,
0142 |           boundaryMessageId: newBoundaryId,
0143 |         }
0144 |       })
0145 |       .filter((cp): cp is NonNullable<typeof cp> => cp !== null)
0146 |
0147 |     return {
0148 |       ...thread,
0149 |       messages: newMessages,
0150 |       createdAt: Date.now(),
0151 |       id: uuidv4(),
0152 |       // Preserve undefined if no compactionPoints, empty array if had some but
0153 |       compactionPoints: newCompactionPoints?.length ? newCompactionPoints : thre
0154 |     }
0155 |   })
0156 | }
0157 |
0158 | export interface KnowledgeBase {
0159 |   id: number

```

```

0160 |   name: string
0161 |   embeddingModel: string
0162 |   parentNode?: string
0163 |   siblingNode?: string
0164 |   documentParser?: DocumentParserConfig
0165 |   createdAt: number
0166 | }
0167
0168 | export interface KnowledgeBaseFile {
0169 |   id: number
0170 |   parentId: number
0171 |   fileName: string
0172 |   filePath: string
0173 |   mime_type: string
0174 |   file_size: number
0175 |   chunk_count: number
0176 |   total_chunks: number
0177 |   status: string
0178 |   error: string
0179 |   createdAt: number
0180 |   parser_type?: DocumentParserType
0181 | }
0182
0183 | export interface KnowledgeBaseSearchResult {
0184 |   id: number
0185 |   score: number
0186 |   text: string
0187 |   fileId: number
0188 |   fileName: string
0189 |   mimeType: string
0190 |   chunkIndex: number
0191 | }
0192
0193 | export type FileMeta = {
0194 |   name: string
0195 |   path: string
0196 |   type: string
0197 |   size: number
0198 | }
0199
0200 | export * from './types/image-generation'
0201 | export * from './types/session'
0202 | export * from './types/settings'
// ===== HKB FILE: src/shared/types.ts =====
// ===== FILE: src/shared/utils/cache.ts =====
0203 | // Cross-platform cache implementation that works in both browser and node.js
0204 | export interface CacheItem {
0205 |   value: T
0206 |   expiresAt: number
0207 | }

```

```

0006 |
0007 | // Memory cache to store ongoing promises and prevent duplicate calls
0008 | const pendingPromises = new Map<string, Promise<unknown>>()
0009 |
0010 | // Memory-only cache that expires on restart
0011 | const memoryCache = new Map<string, CacheItem<unknown>>()
0012 |
0013 | // Cross-platform storage adapter
0014 | class LocalStorageAdapter {
0015 |     private name: string
0016 |     private memoryFallback = new Map<string, string>()
0017 |
0018 |     constructor(name: string) {
0019 |         this.name = name
0020 |     }
0021 |
0022 |     async getItem(key: string): Promise<string | null> {
0023 |         // In renderer process with localforage
0024 |         if (typeof window !== 'undefined' && 'localforage' in window) {
0025 |             try {
0026 |                 const localforage = (await import('localforage')).default
0027 |                 const store = localforage.createInstance({ name: this.name })
0028 |                 return await store.getItem<string>(key)
0029 |             } catch (error) {
0030 |                 console.error('Error accessing localforage:', error)
0031 |                 return this.memoryFallback.get(key) || null
0032 |             }
0033 |         }
0034 |
0035 |         // In main process or fallback
0036 |         return this.memoryFallback.get(key) || null
0037 |     }
0038 |
0039 |     async setItem(key: string, value: string): Promise<void> {
0040 |         // In renderer process with localforage
0041 |         if (typeof window !== 'undefined' && 'localforage' in window) {
0042 |             try {
0043 |                 const localforage = (await import('localforage')).default
0044 |                 const store = localforage.createInstance({ name: this.name })
0045 |                 await store.setItem(key, value)
0046 |                 return
0047 |             } catch (error) {
0048 |                 console.error('Error accessing localforage:', error)
0049 |                 this.memoryFallback.set(key, value)
0050 |                 return
0051 |             }
0052 |         }
0053 |
0054 |         // In main process or fallback
0055 |         this.memoryFallback.set(key, value)

```

```

0056 |   }
0057 |
0058 |   async removeItem(key: string): Promise<void> {
0059 |     // In renderer process with localforage
0060 |     if (typeof window !== 'undefined' && 'localforage' in window) {
0061 |       try {
0062 |         const localforage = (await import('localforage')).default
0063 |         const store = localforage.createInstance({ name: this.name })
0064 |         await store.removeItem(key)
0065 |         return
0066 |       } catch (error) {
0067 |         console.error('Error accessing localforage:', error)
0068 |         this.memoryFallback.delete(key)
0069 |         return
0070 |       }
0071 |     }
0072 |
0073 |     // In main process or fallback
0074 |     this.memoryFallback.delete(key)
0075 |   }
0076 | }
0077 |
0078 | export const store = new CrossPlatformStorage('chatboxcache')
0079 |
0080 | async function cacheWithStorage<T>({
0081 |   key: string,
0082 |   getter: () => Promise<T>,
0083 |   options: {
0084 |     ttl: number // 缓存过期时间, 单位为毫秒
0085 |     refreshFallbackToCache?: boolean // 如果刷新时获取新值失败, 是否从缓存中继续使用过期的旧值
0086 |     memoryOnly?: boolean // 是否仅使用内存缓存
0087 |   }
0088 | }): Promise<T> {
0089 |   let cache: CacheItem<T> | null = null
0090 |
0091 |   if (options.memoryOnly) {
0092 |     cache = (memoryCache.get(key) as CacheItem<T> | undefined) || null
0093 |   } else {
0094 |     const cachedStr = await store.getItem(key)
0095 |     if (cachedStr) {
0096 |       try {
0097 |         cache = JSON.parse(cachedStr)
0098 |       } catch (e) {
0099 |         console.error(`Error parsing cache for key ${key}:`, e)
0100 |       }
0101 |     }
0102 |   }
0103 |
0104 |   if (cache && cache.expireAt > Date.now()) {
0105 |     return cache.value

```



```

0106 |   }
0107 |
0108 |   // Check if there's already a pending promise for this key
0109 |   const existingPromise = pendingPromises.get(key) as Promise<T> | undefined
0110 |   if (existingPromise) {
0111 |     return existingPromise
0112 |   }
0113 |
0114 |   // Create new promise and store it to prevent duplicate calls
0115 |   const promise = (async () => {
0116 |     try {
0117 |       const newVals = await getter()
0118 |       const newCache: CacheItem<T> = {
0119 |         value: newVals,
0120 |         expiresAt: Date.now() + options.ttl,
0121 |       }
0122 |
0123 |       if (options.memoryOnly) {
0124 |         memoryCache.set(key, newCache)
0125 |       } else {
0126 |         await store.setItem(key, JSON.stringify(newCache))
0127 |       }
0128 |
0129 |       return newVals
0130 |     } catch (e) {
0131 |       if (options.refreshFallbackToCache && cache) {
0132 |         return cache.value
0133 |       }
0134 |       throw e
0135 |     } finally {
0136 |       // Removes the promise from pending map when done
0137 |       pendingPromises.delete(key)
0138 |     }
0139 |   })()
0140 |
0141 |   pendingPromises.set(key, promise)
0142 |   return promise
0143 | }
0144 |
0145 | export async function cache<T>({
0146 |   key: string,
0147 |   getter: () => Promise<T>,
0148 |   options: {
0149 |     ttl: number // 缓存过期时间, 单位为毫秒
0150 |     refreshFallbackToCache?: boolean // 如果刷新时获取新值失败, 是否从缓存中继续使用过期值
0151 |     memoryOnly?: boolean // 是否仅使用内存缓存
0152 |   },
0153 | }: Promise<T>): T {
0154 |   return cacheWithStorage(key, getter, options)
0155 | }

```

```

// ===== END FILE: src/shared/utils/cache.ts =====
// ===== FILE: src/shared/utils/index.ts =====
0001 | export * from './json_utils'
0002 | export * from './llm_utils'
0003 | export * from './network_utils'
0004 | export * from './word_count'
0005 |
0006 | // Format numbers with K/M suffixes (for tokens, file sizes, etc.)
0007 | export function formatNumber(num: number, unit = ''): string {
0008 |   if (num === 0) return `0${unit ? ` ${unit}` : ''}`
0009 |
0010 |   if (num >= 1_000_000) {
0011 |     return `${(num / 1_000_000).toFixed(1)}M${unit ? ` ${unit}` : ''}`
0012 |   }
0013 |   if (num >= 1_000) {
0014 |     return `${(num / 1_000).toFixed(0)}K${unit ? ` ${unit}` : ''}`
0015 |   }
0016 |   return `${num}${unit ? ` ${unit}` : ''}`
0017 | }
0018 |
0019 | // Format file sizes with proper binary units (1024-based)
0020 | export function formatFileSize(bytes: number): string {
0021 |   if (bytes === 0) return '0 B'
0022 |   const k = 1024
0023 |   const sizes = ['B', 'KB', 'MB', 'GB', 'TB', 'PB', 'EB', 'ZB', 'YB']
0024 |   const i = Math.floor(Math.log(bytes) / Math.log(k))
0025 |   return `${parseFloat((bytes / k ** i).toFixed(1))} ${sizes[i]}`
0026 | }
// ===== END FILE: src/shared/utils/index.ts =====
// ===== FILE: src/shared/utils/json_utils.ts =====
0001 | /**
0002 |  * 解析 JSON 字符串, 如果解析失败则返回空对象
0003 |  * @param json 要解析的 JSON 字符串
0004 |  * @returns 解析后的对象, 解析失败时返回空对象
0005 |  */
0006 | export function parseJsonOrEmpty(json: string): any {
0007 |   try {
0008 |     return JSON.parse(json)
0009 |   } catch (e) {
0010 |     return {}
0011 |   }
0012 | }
// ===== END FILE: src/shared/utils/json_utils.ts =====
// ===== FILE: src/shared/utils/knowledge-base-model-parser.ts =====
0001 | /**
0002 |  * Parse knowledge base model string format: "providerId:modelId"
0003 |  * Handles model IDs that contain colons by using the first colon as separator
0004 |  */
0005 | export function parseKnowledgeBaseModelString(modelString: string): { providerId
0006 |   if (!modelString) return null

```

```

0007 |
0008 |     const colonIndex = modelString.indexOf(':')
0009 |     if (colonIndex === -1) return null
0010 |
0011 |     const providerId = modelString.substring(0, colonIndex)
0012 |     const modelId = modelString.substring(colonIndex + 1)
0013 |
0014 |     if (!providerId || !modelId) return null
0015 |
0016 |     return { providerId, modelId }
0017 | }
// ===== END FILE: src/shared/utils/knowledge-base-model-parser.ts =====
// ===== FILE: src/shared/utils/llm_utils.ts =====
0001 | import { ModelProviderEnum } from '../types';
0002 |
0003 | export function normalizeOpenAIApiHostAndPath(
0004 |     options: { apiHost?: string; apiPath?: string },
0005 |     defaults?: { apiHost?: string; apiPath?: string }
0006 | ) {
0007 |     let { apiHost, apiPath } = options
0008 |     if (apiHost) {
0009 |         apiHost = apiHost.trim()
0010 |     }
0011 |     if (apiPath) {
0012 |         apiPath = apiPath.trim()
0013 |     }
0014 |     const DEFAULT_HOST = defaults?.apiHost ?? 'https://api.openai.com/v1'
0015 |     const DEFAULT_PATH = defaults?.apiPath ?? '/chat/completions'
0016 |     // 如果 apiHost 为空, 直接返回默认的 apiHost 和 apiPath
0017 |     if (!apiHost) {
0018 |         apiHost = DEFAULT_HOST
0019 |         apiPath = DEFAULT_PATH
0020 |         return { apiHost, apiPath }
0021 |     }
0022 |     // 处理前后 '/' 的干扰
0023 |     if (apiHost.endsWith('/')) {
0024 |         apiHost = apiHost.slice(0, -1)
0025 |     }
0026 |     if (apiPath && !apiPath.startsWith('/')) {
0027 |         apiPath = '/' + apiPath
0028 |     }
0029 |     // https 协议
0030 |     if (apiHost && !apiHost.startsWith('http://') && !apiHost.startsWith('https://')) {
0031 |         apiHost = 'https://' + apiHost
0032 |     }
0033 |     // 如果用户在配置中配置了完整的 URL, 则不需要再添加协议和路径
0034 |     // 可以兼容的输入情况有:
0035 |     //   apiHost=https://my.proxy.com/v1/chat/completions
0036 |     if (apiHost.endsWith(DEFAULT_PATH)) {
0037 |         apiHost = apiHost.replace(DEFAULT_PATH, '')

```

```

0038 |     apiPath = DEFAULT_PATH
0039 | }
0040 | // 如果当前配置的是 OpenAI 的 API, 统一为默认的 apiHost 和 apiPath
0041 | if (apiHost.endsWith('/:api.openai.com') || apiHost.endsWith('/:api.openai.c
0042 |     apiHost = DEFAULT_HOST
0043 |     apiPath = DEFAULT_PATH
0044 |     return { apiHost, apiPath }
0045 | }
0046 | // 如果当前配置的是 OpenRouter 的 API, 统一 apiHost 和 apiPath
0047 | if (apiHost.endsWith('/:openrouter.ai') || apiHost.endsWith('/:openrouter.ai
0048 |     apiHost = 'https://openrouter.ai/api/v1'
0049 |     apiPath = DEFAULT_PATH
0050 |     return { apiHost, apiPath }
0051 | }
0052 | // 如果当前配置的是 x 的 API, 统一 apiHost 和 apiPath
0053 | if (apiHost.endsWith('/:api.x.com') || apiHost.endsWith('/:api.x.com/v1')) {
0054 |     apiHost = 'https://api.x.com/v1'
0055 |     apiPath = DEFAULT_PATH
0056 |     return { apiHost, apiPath }
0057 | }
0058 | // 如果只配置 apiHost, 且 apiHost 不以 /v1 结尾
0059 | if (!apiHost.endsWith('/v1') && !apiPath) {
0060 |     apiHost = apiHost + '/v1'
0061 |     apiPath = DEFAULT_PATH
0062 | }
0063 | if (!apiPath) {
0064 |     apiPath = DEFAULT_PATH
0065 | }
0066 | return { apiHost, apiPath }
0067 | }
0068 |
0069 | export function normalizeOpenAIResponsesHostAndPath(options: { apiHost?: string;
0070 |     const trimmedApiPath = options.apiPath?.trim()
0071 |     const hasCustomApiPath = !!trimmedApiPath && trimmedApiPath !== '/responses'
0072 |     const normalized = normalizeOpenAIApiHostAndPath(
0073 |         hasCustomApiPath ? { ...options, apiPath: trimmedApiPath } : { ...options, a
0074 |         { apiPath: '/responses' }
0075 |     )
0076 |
0077 |     if (!hasCustomApiPath) {
0078 |         normalized.apiPath = '/responses'
0079 |     }
0080 |
0081 |     return normalized
0082 | }
0083 |
0084 | export function normalizeClaudeHost(apiHost: string) {
0085 |     apiHost = apiHost.trim()
0086 |     if (apiHost === 'https://api.anthropic.com') {
0087 |         apiHost = `${apiHost}/v1`

```

```

0088 |   }
0089 |   if (apiHost.endsWith('/')) {
0090 |     apiHost = apiHost.slice(0, apiHost.length - 1)
0091 |   }
0092 |   return {
0093 |     apiHost,
0094 |     apiPath: '/messages',
0095 |   }
0096 | }
0097 |
0098 | export function normalizeGeminiHost(apiHost: string) {
0099 |   apiHost = apiHost.trim()
0100 |   if (apiHost.endsWith('/')) {
0101 |     apiHost = apiHost.slice(0, apiHost.length - 1)
0102 |   }
0103 |   apiHost = `${apiHost}/v1beta`
0104 |   return {
0105 |     apiHost,
0106 |     apiPath: '/models/[model]',
0107 |   }
0108 | }
0109 |
0110 | export function normalizeAzureEndpoint(endpoint: string) {
0111 |   let origin = endpoint
0112 |   try {
0113 |     origin = new URL(endpoint.trim()).origin
0114 |   } catch (_error) {
0115 |     origin = `https://${origin}.openai.azure.com`
0116 |   }
0117 |   return {
0118 |     endpoint: `${origin}/openai`,
0119 |     apiPath: '/chat/completions',
0120 |   }
0121 | }
0122 |
0123 | export function isOpenAICompatible(providerId: string, _modelId: string) {
0124 |   return (
0125 |     [
0126 |       ModelProviderEnum.OpenAI,
0127 |       ModelProviderEnum.Wanjie,
0128 |       ModelProviderEnum.SiliconFlow,
0129 |       ModelProviderEnum.OpenRouter,
0130 |       ModelProviderEnum.Ollama,
0131 |       ModelProviderEnum.ChatGLM6B,
0132 |       ModelProviderEnum.XAI,
0133 |       ModelProviderEnum.Groq,
0134 |       ModelProviderEnum.DeepSeek,
0135 |       ModelProviderEnum.LMStudio,
0136 |     ].includes(providerId as ModelProviderEnum) || providerId.startsWith('custom
0137 |   )

```



```

0138 | }
// ===== END FILE: src/shared/utils/llm_utils.ts =====
// ===== FILE: src/shared/utils/message.ts =====
0001 | import { assign, cloneDeep, omit } from 'lodash'
0002 | import type { Message, MessageContentParts, MessagePicture, SearchResultItem } f
0003 | import { countWord } from './word_count'
0004 |
0005 | export function getMessageText(message: Message, includeImagePlaceholder = true,
0006 |   if (message.contentParts && message.contentParts.length > 0) {
0007 |     return message.contentParts
0008 |       .map((c) => {
0009 |         if (c.type === 'reasoning') {
0010 |           return includeReasoning ? c.text : null
0011 |         }
0012 |         if (c.type === 'text') {
0013 |           return c.text
0014 |         }
0015 |         if (c.type === 'image') {
0016 |           return includeImagePlaceholder ? '[image]' : null
0017 |         }
0018 |         return ''
0019 |       })
0020 |       .filter((c) => c !== null)
0021 |       .join('\n')
0022 |   }
0023 |   return ''
0024 | }
0025 |
0026 | // 只有这里可以访问 message 的 content / webBrowsing 字段, 迁移到 contentParts 字段
0027 | export function migrateMessage(
0028 |   message: Omit<Message, 'contentParts'> & { contentParts?: MessageContentParts
0029 | ): Message {
0030 |   const result: Message = {
0031 |     id: message.id || '',
0032 |     role: message.role || 'user',
0033 |     contentParts: message.contentParts || [],
0034 |   }
0035 |   // 还是保留原始 content 字段, 删除 webBrowsing 字段
0036 |   assign(result, omit(message, 'webBrowsing'))
0037 |
0038 |   // 如果 contentParts 不存在, 或者 contentParts 为空, 或者 contentParts 的内容为 '...' (placeholder
0039 |   if (
0040 |     (!result.contentParts?.length || getMessageText(result) === '...' || !getMes
0041 |     'content' in message
0042 |   ) {
0043 |     const imageParts = (message as Message & { pictures?: MessagePicture[] }).pictures
0044 |       ?.filter((pic) => pic.storageKey || pic.url)
0045 |       .map((pic) => ({ type: 'image' as const, storageKey: pic.storageKey!,
0046 |         result.contentParts = [{ type: 'text', text: String(message.content ?? '
0047 |   }

```

```

0048 |
0049 |   if ('webBrowsing' in message) {
0050 |     const webBrowsing = message.webBrowsing as {
0051 |       query: string[]
0052 |       links: { title: string; url: string }[]
0053 |     }
0054 |     result.contentParts.unshift({
0055 |       type: 'tool-call',
0056 |       state: 'result',
0057 |       toolCallId: `web_search_${message.id}`,
0058 |       toolName: 'web_search',
0059 |       args: {
0060 |         query: webBrowsing.query.join(' ', ''),
0061 |       },
0062 |       result: {
0063 |         query: webBrowsing.query.join(' ', ''),
0064 |         searchResults: webBrowsing.links.map((link) => ({
0065 |           title: link.title,
0066 |           link: link.url,
0067 |           snippet: link.title,
0068 |         })) satisfies SearchResultItem[],
0069 |       },
0070 |     })
0071 |   }
0072 |
0073 |   return result
0074 | }
0075 |
0076 | export function cloneMessage(message: Message): Message {
0077 |   return cloneDeep(message)
0078 | }
0079 |
0080 | export function isEmptyMessage(message: Message): boolean {
0081 |   return getMessageText(message, true, true).length === 0 && !message.files?.len
0082 | }
0083 |
0084 | export function countMessageWords(message: Message): number {
0085 |   return countWord(getMessageText(message))
0086 | }
0087 |
0088 | export function mergeMessages(a: Message, b: Message): Message {
0089 |   const ret = cloneMessage(a)
0090 |   // Merge contentParts
0091 |   ret.contentParts = [...(ret.contentParts || []), ...(b.contentParts || [])]
0092 |
0093 |   return ret
0094 | }
0095 |
0096 | export function fixMessageRoleSequence(messages: Message[]): Message[] {
0097 |   let result: Message[] = []

```

```

0098 |   if (messages.length <= 1) {
0099 |     result = messages
0100 |   } else {
0101 |     let currentMessage = cloneMessage(messages[0]) // 复制, 避免后续修改导致的引用问题
0102 |
0103 |     for (let i = 1; i < messages.length; i++) {
0104 |       const message = cloneMessage(messages[i]) // 复制消息避免修改原对象
0105 |
0106 |       if (message.role === currentMessage.role) {
0107 |         currentMessage = mergeMessages(currentMessage, message)
0108 |       } else {
0109 |         result.push(currentMessage)
0110 |         currentMessage = message
0111 |       }
0112 |     }
0113 |     result.push(currentMessage)
0114 |   }
0115 |   // 如果顺序中的第一条 assistant 消息前面不是 user 消息, 则插入一个 user 消息
0116 |   const firstAssistantIndex = result.findIndex((m) => m.role === 'assistant')
0117 |   if (firstAssistantIndex !== -1 && result[firstAssistantIndex - 1]?.role !== 'u
0118 |     result = [
0119 |       ...result.slice(0, firstAssistantIndex),
0120 |       { role: 'user', contentParts: [{ type: 'text', text: 'OK.' }], id: 'user_b
0121 |       ...result.slice(firstAssistantIndex),
0122 |     ]
0123 |   }
0124 |   return result
0125 | }
0126 |
0127 | /**
0128 |  * SequenceMessages organizes and orders messages to follow the sequence: system
0129 |  * 这个方法只能用于 llm 接口请求前的参数构造, 因为会过滤掉消息中的无关字段, 所以不适用于其他消息存储的场景
0130 |  * 这个方法本质上是 golang API 服务中方法的 TypeScript 实现
0131 |  * @param msgs
0132 |  * @returns
0133 |  */
0134 | export function sequenceMessages(msgs: Message[]): Message[] {
0135 |   // Merge all system messages first
0136 |   let system: Message = {
0137 |     id: '',
0138 |     role: 'system',
0139 |     contentParts: [],
0140 |   }
0141 |   for (const msg of msgs) {
0142 |     if (msg.role === 'system') {
0143 |       system = mergeMessages(system, msg)
0144 |     }
0145 |   }
0146 |   // Initialize the result array with the non-empty system message, if present
0147 |   const ret: Message[] = !isEmptyMessage(system) ? [system] : []

```

```

0148 |   let next: Message = {
0149 |     id: '',
0150 |     role: 'user',
0151 |     contentParts: [],
0152 |   }
0153 |   let isFirstUserMsg = true // Special handling for the first user message
0154 |   for (const msg of msgs) {
0155 |     // Skip the already processed system messages or empty messages
0156 |     if (msg.role === 'system' || isEmptyMessage(msg)) {
0157 |       continue
0158 |     }
0159 |     // Merge consecutive messages from the same role
0160 |     if (msg.role === next.role) {
0161 |       next = mergeMessages(next, msg)
0162 |       continue
0163 |     }
0164 |     // Merge all assistant messages as a quote block if constructing the first u
0165 |     if (isEmptyMessage(next) && isFirstUserMsg && msg.role === 'assistant') {
0166 |       const text = getMessageText(msg)
0167 |       // Split and quote each line, preserving empty lines
0168 |       const lines = text.split('\n')
0169 |       // Remove the last empty element only if text ends with newline
0170 |       const linesToQuote = text.endsWith('\n') ? lines.slice(0, -1) : lines
0171 |       const quotedText = linesToQuote.map(line => `> ${line}`.trim()).join('\n')
0172 |       // Add back the ending newline(s) to match original structure
0173 |       const quote = quotedText.endsWith('\n') ? quotedText : quotedText + '\n'
0174 |       // Place the message to avoid matching the original, which can cause
0175 |       // the first ">" prefixes if responsesMessages is called multiple times
0176 |       const quotedMsg = cloneMessage(msg)
0177 |       quotedMsg.contentParts = [type: 'text', text: quote]
0178 |       next = mergeMessages(next, quotedMsg)
0179 |       continue
0180 |     }
0181 |     // If not the first user message, add the current message to the result and
0182 |     if (!isEmptyMessage(next)) {
0183 |       ret.push(next)
0184 |       isFirstUserMsg = false
0185 |     }
0186 |     next = msg
0187 |   }
0188 |   // Add the last message if it's not empty
0189 |   if (!isEmptyMessage(next)) {
0190 |     ret.push(next)
0191 |   }
0192 |   // If there's only one system message, convert it to a user message
0193 |   if (ret.length === 1 && ret[0].role === 'system') {
0194 |     ret[0].role = 'user'
0195 |   }
0196 |   return ret
0197 | }

```

```

// ===== END FILE: src/shared/utils/message.ts =====
// ===== FILE: src/shared/utils/model_settings.ts =====
0001 | import type { Settings } from '../types'
0002 |
0003 | export function getModelSettings(globalSettings: Settings, providerId: string, m
0004 |     const providerEntry = Object.entries(globalSettings.providers ?? {}).find(([ke
0005 |     if (!providerEntry) {
0006 |         const error = new Error(`provider ${providerId} not set`)
0007 |
0008 |         throw error
0009 |     }
0010 |
0011 |     return {
0012 |         ...globalSettings,
0013 |         provider: providerId,
0014 |         modelId,
0015 |     }
0016 | }
// ===== END FILE: src/shared/utils/model_settings.ts =====
// ===== FILE: src/shared/utils/network_utils.ts =====
0001 | /**
0002 |  * 检查 URL 是否指向本地主机
0003 |  * @param url 要检查的 URL
0004 |  * @returns 如果是本地主机则返回 true
0005 |  */
0006 | export function isLocalHost(url: string): boolean {
0007 |     try {
0008 |         const { hostname } = new URL(url)
0009 |         if (hostname === 'localhost' || hostname === '[::1]' || hostname === ':::1')
0010 |             if (hostname.startsWith('127.')) return true
0011 |             if (hostname.startsWith('192.168.')) return true
0012 |             if (hostname.startsWith('10.')) return true
0013 |             // 仅匹配 172.16.0.0 - 172.31.255.255
0014 |             const match = hostname.match(/^172\.(\\d{1,3})\\.\\/))
0015 |             if (match) {
0016 |                 const second = Number(match[1])
0017 |                 return second >= 16 && second <= 31
0018 |             }
0019 |         } catch (_) {
0020 |             /* ignore malformed URL */
0021 |             return false
0022 |         }
0023 |     }
0024 | }
// ===== END FILE: src/shared/utils/network_utils.ts =====
// ===== FILE: src/shared/utils/sentry_adapter.ts =====
0001 | /**
0002 |  * 跨平台 Sentry 适配器接口
0003 |  * 允许在不同环境中使用统一的错误上报 API
0004 |  */

```



```

0005 | export interface SentryAdapter {
0006 |   captureException(error: any): void
0007 |   withScope(callback: (scope: SentryScope) => void): void
0008 | }
0009 |
0010 | export interface SentryScope {
0011 |   setTag(key: string, value: string): void
0012 |   setExtra(key: string, value: any): void
0013 | }
// ===== END FILE: src/shared/utils/sentry_adapter.ts =====
// ===== FILE: src/shared/utils/word_count.ts =====
0001 | /**
0002 |  * Word Count
0003 |  *
0004 |  * Word count in respect of CJK characters.
0005 |  *
0006 |  * Copyright (c) 2015 - 2016 by Hsiaoming Yang.
0007 |  *
0008 |  * https://github.com/yuehu/word-count
0009 |  */
0010 | const pattern =
0011 |   /[\p{Z}\p{L}\p{N}]/u
0012 |
0013 | /**
0014 |  * 统计文本中的字数, 支持中日韩字符
0015 |  * @param data 要统计的文本
0016 |  * @returns 字数
0017 |  */
0018 | export function countWord(data: string): number {
0019 |   try {
0020 |     data = typeof data === 'string' ? data : JSON.stringify(data)
0021 |     const m = data.match(pattern)
0022 |     let count = 0
0023 |     if (!m) {
0024 |       return 0
0025 |     }
0026 |     for (let i = 0; i < m.length; i++) {
0027 |       if (m[i].charCodeAt(0) >= 0x4e00) {
0028 |         count += m[i].length
0029 |       } else {
0030 |         count += 1
0031 |       }
0032 |     }
0033 |     return count
0034 |   } catch (e) {
0035 |     // 在共享层不使用 Sentry, 简单返回 -1 表示错误
0036 |     return -1
0037 |   }
0038 | }
// ===== END FILE: src/shared/utils/word_count.ts =====

```